OpenMarkX: An Extensible Platform for Watermarking Open-Source Model and Agent Outputs

Jiaqi Xue[1]*, **Yifei Zhao**[1]*, **Mansour Al Ghanim**[1], **Shangqian Gao**[2]
Ruimin Sun[3], **Qian Lou**[1], **Mengxin Zheng**[1]
[1]LoopLeap Lab, University of Central Florida
[2]Florida State University, [3]Florida International University
{jiaqi.xue,yifei.zhao,qian.lou,mengxin.zheng}@ucf.edu

Website https://github.com/loopleap-lab/OpenMarkX#website
Code https://github.com/loopleap-lab/OpenMarkX

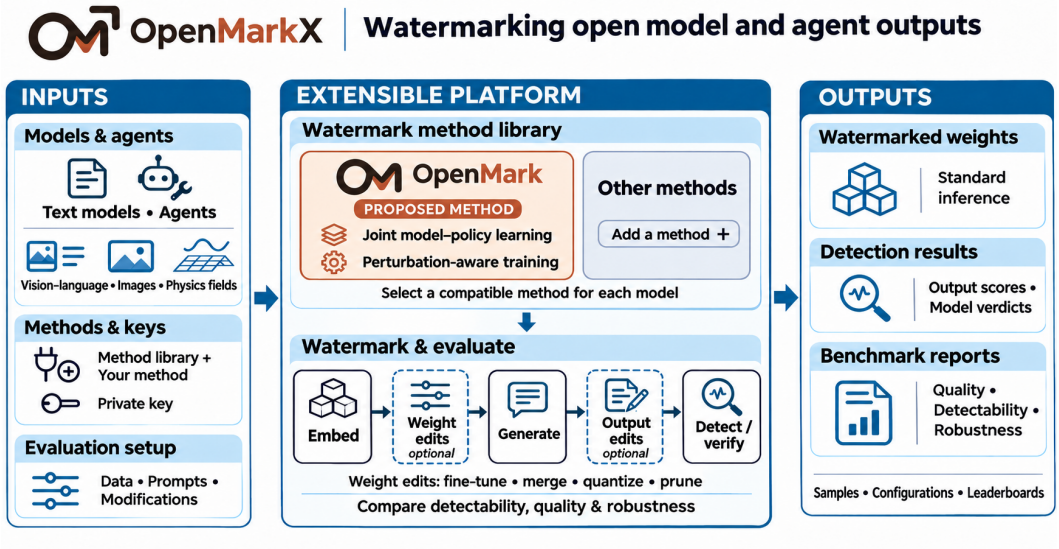


Figure 1: **Overview of OpenMarkX.** Configurable models and watermark methods feed embedding, generation, and evaluation. **OpenMark** supplies joint learning and perturbation-aware training.

Abstract

We introduce **OpenMarkX**, an extensible platform for watermarking open-source model and agent outputs under standard inference. It takes models, compatible methods, private keys, and evaluation settings, and produces watermarked weights, detection results, and quality and robustness reports. Its 16 text-watermark configurations are complemented by agent, vision-language, image, and physics-field workflows. Our proposed method, **OpenMark**, addresses *learnability* and *robustness* through joint model–policy learning and perturbation-aware training. On several open-source LLMs, **OpenMark** achieves 0.92 TPR at 1% FPR while reducing perplexity by 20.5% on average relative to the strongest prior method, and remains more detectable after such modifications, e.g., retaining 0.57 TPR at 5% FPR after 0.5-ratio model merging, compared with at most 0.34 for prior methods.

*Equal contribution.

40th Conference on Neural Information Processing Systems (NeurIPS 2026).

1 Introduction

As language models enter applications [1–7], their outputs can circulate long after generation. Concerns about misuse [8–13] motivate a provenance question: can the owner of a released model recognize its outputs? A watermark embeds a statistical signal that the owner can test for later [14–17], providing evidence of model origin for attribution and IP protection [18, 19].

Figure 1 introduces **OPENMARKX**, a platform for embedding and evaluating watermarks in open-source model and agent outputs. Here, open-source means that downstream users can access the weights and control inference. Our experiments focus on text, including an agent's final answer. We also demonstrate compatible methods on vision-language, image, and physics models.

Figure 2 illustrates why access to the weights matters. In panel (a), a user sends a prompt to a closed-source service. The provider controls decoding and uses a secret rule to favor certain tokens [20–22]. A detector can later test the returned text for that preference. In panel (b), the user downloads the weights and chooses the decoder. The user can omit an external watermarking step, so the owner trains the signal into the weights before release. Ordinary decoding then produces text that the owner can check.

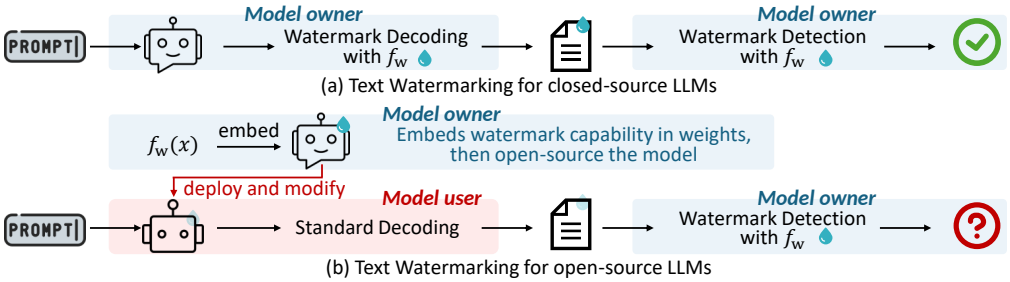


Figure 2: Text watermarking for (a) closed-source and (b) open-source LLMs. Closed-source watermarking relies on watermark decoding, while open-source watermarking requires embedding the watermark into the model weights so that standard decoding still produces watermarked text.

An owner can add watermark logits to the output-layer bias [23], but many open models omit that bias, making the added component easy to locate and remove. Learning-based methods instead train the model to reproduce a watermarked teacher's outputs [24, 25].

Figure 3 shows two difficulties. First, a strong teacher signal is easier to detect but can reduce generation quality: increasing KGW strength from $\delta = 1$ to 2 raises AUC from 0.84 to 0.99 and PPL from 4.1 to 5.0. The decoding-based comparison achieves AUC > 0.9 with PPL < 4.5. Longer prefixes make the rule harder to approximate [21, 26], yet prior learning-based schemes typically use $n \leq 1$ [24–26]: even at $\delta = 2$, the learned 2-gram KGW watermark reaches only 0.54 AUC. Second, editing the released weights weakens the signal: at a merge ratio of 0.5, the prior method reaches 0.79 AUC and **OPENMARK** retains 0.87.

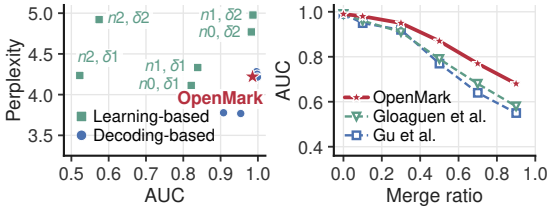


Figure 3: *(Left)* learning-based and decoding-based KGW watermarks under varying watermark hyperparameters n and δ. *(Right)* existing learning-based watermarks' AUC after merging with the unwatermarked LLM.

Our method, **OPENMARK**, jointly trains the generator and the watermark policy used for detection. This addresses a mismatch in distillation methods [24, 25]: a student may only partially learn a fixed rule such as KGW [21], while the detector still tests for the original rule. Joint training lets the policy adapt to patterns the generator can learn.

To improve robustness, **OPENMARK** also trains against a weight perturbation estimated from anti-watermarked text. Earlier work selects parameters that remain stable during fine-tuning on a particular dataset [27]. Our objective directly penalizes loss of the watermark along an estimated erasure direction (Section 4).

We compare **OPENMARK** with prior methods [24, 27] on LLaMA3-8B, Phi2-2.7B, and LLaMA3.2-3B. After 300 and 1,500 fine-tuning steps, it retains TPRs of 0.70 and 0.37 at 5% FPR, versus baseline maxima of 0.48 and 0.23. Section 3 describes the platform, and Section 5 evaluates detection, generation quality, and robustness across edits. Table 4 reports the four demonstrations beyond standalone text generation.

2 Related Work and Threat Model

2.1 Text watermarks for Closed-source LLMs

To ensure traceability of content generated by LLMs, watermarking techniques have been proposed to embed identifiable statistical signals into model outputs. Among these, *decoding-based watermarking* [21, 23, 28, 29] is a widely adopted approach. The watermark decoding function $f_w(\pi_\theta(x), \xi)$ leverages a secret key ξ to transform the original next-token distribution $\pi_\theta(\cdot \mid x)$ into a modified distribution that embeds a detectable watermark signal in the generated text. This enables post-hoc detection via a test function $f_d(x, \xi)$, which computes a p-value indicating the presence of a watermark. However, decoding-based watermarking relies on customized decoding algorithms and is not applicable in open-model settings where users control the decoding process.

2.2 Text watermarks for Open-source LLMs

Current watermarking schemes for open-source language models fall into two types: *Input Prompt-dependent* and *Input Prompt-independent*. The former requires the input prompt for detection, while the latter detects watermarks from output text alone. Notably, **OPENMARK** is input prompt-independent.

2.2.1 Input Prompt-dependent

Several recent works propose open-source watermarking techniques that require access to the input prompt for detection. [30] jointly train a detector and an LLM to embed detectable signals into generated text, while [31] perturbs model weights with Gaussian noise and detects watermarks using gradient-based statistical tests. Both approaches rely on the input prompt during detection, which limits their practicality in real-world settings.

2.2.2 Input Prompt-independent

We further divide input prompt-independent watermarking methods into two classes: learning-based watermarks and structural-editing watermarks.

Learning-based Watermark. [24] shows that decoding-based watermarks can be embedded into model weights by distilling the watermark from a teacher LLM π_o. Then, the same watermark scheme can be used to detect the watermark in the student LLM π_θ. The teacher generates watermarked data $\mathcal{D}_{\text{wm}}$, which the student fine-tunes using the cross-entropy loss:

$$\mathcal{L}_{\text{sampling}}(\pi_\theta) = \mathbb{E}_{x \sim \mathcal{D}_{\text{wm}}} \big[\sum_{t=1}^{|x|} - \log \pi_\theta(x_t | x_{<t}) \big] \tag{1}$$

Alternatively, by exploiting the logits, the student can be fine-tuned to mimic the teacher LLM's next-token distribution using the KL-divergence loss:

$$\mathcal{L}_{\text{logit}}(\pi_\theta) = \mathbb{E}_{x \sim \mathcal{D}} \big[\sum_{t=1}^{|x|} \text{KL}(f_w(\pi_o(\cdot \mid x_{<t}), \xi_w) \parallel \pi_\theta(\cdot \mid x_{<t})) \big] \tag{2}$$

Structural-editing Watermarks. [23] embed watermarks by adding small, token-specific Gaussian perturbations to the model's output-layer bias. Detection aggregates the bias perturbations of each output token. As most open-source LLMs disable output-layer biases by default[2], implementing this method requires explicitly enabling the bias term, introducing an architectural modification. Such changes remain identifiable and removable by analyzing and modifying the model architecture. We therefore focus on learning-based methods as the closest comparison to **OPENMARK**, since they share the goal of making standard decoding emit watermarked text without architectural modifications.

[2]For example, Hugging Face Transformers defines the output layers of LLaMA, Qwen, and Mistral with `bias=False`.

2.3 Agent Output and Behavioral Watermarking

Agent watermarking can target generated content or the behavior that produces it. AgentMark embeds identifiers in planning decisions using distribution-preserving sampling [32]; ActHook embeds key-triggered hook actions in agent trajectory datasets [33]; and SeqWM watermarks history-conditioned behavioral transitions [34]. These approaches address action or trajectory provenance. The current **OPENMARKX** agent workflow instead applies a compatible content watermark to an open language-model backbone and detects its signal in the agent's final generated answer. This scope complements behavioral watermarking and does not claim to authenticate every tool call or the entire execution trajectory.

2.4 Threat Model

The following threat model governs the **OPENMARK** method and its text-generation evaluation. **OPENMARKX** additionally records each selected method's detector-access requirements and evaluation configuration, rather than assuming that every method has the same access contract. We consider a threat model where the LLM owner publicly releases the watermarked model weights while keeping both the unwatermarked teacher π_o and detector M private [35–44], as is standard in closed-source watermarks where the watermark seed is also private [21]. The owner's goal is model-level provenance: verifying whether a given text carries the watermark of the released model. Detection is performed solely on the generated text, without access to the user's input prompt or decoding process. With full white-box access to the released model, the user can try to evade detection at two complementary levels: (i) the *text level* (paraphrasing or rewriting generated outputs, as in closed-source watermark attacks [45, 46]), and (ii) the *model-weights level* (fine-tuning, merging, quantization, or pruning the released model) [47–52].

3 The OPENMARKX Platform

A run in Figure 1 starts with a model or agent backbone, a watermark method, a private key, and evaluation data. Each configuration specifies the output unit the detector can score, from a token in a final answer to a value in a predicted physical field.

Extending a run. A model adapter loads the backbone and generates outputs. A watermark module defines how to embed the signal and score those outputs, and an edit module modifies weights or generated samples. A new backbone needs a wrapper and a compatible watermark module integrated with the stages it will run. For example, the ReAct configuration tests final-answer text, whereas the physics detector compares predicted fields with a reference prediction.

Method and edit compatibility. The text benchmark has 16 configurations, including **OPENMARK** variants, 12 decoding-time watermarks from MarkLLM [53] distilled into weights [24], and GaussMark [31]. Table 4 includes WMAR [54] for image tokens and Ripple for physics fields. Each configuration selects applicable stages. A method that fails to learn a detectable signal does not complete the robustness evaluation.

Evaluation and outputs. The owner releases the watermarked weights for standard inference and retains the key and detector artifacts. A baseline run measures detectability and output quality. To test robustness, the evaluator edits weights before generation (fine-tuning, merging, quantization, or pruning) or edits outputs before detection. Run records include the configuration, samples, detection scores, model-verification results, and comparative reports.

4 OPENMARK: The Proposed Watermarking Method

This section describes **OPENMARK**, the proposed text-watermarking method within **OPENMARKX**. Its learned policy and detector address generation–detection consistency, while perturbation-aware training improves resistance to weight modifications. The platform's other methods retain their own embedding and detection mechanisms.

Learning-based watermarking trains an LLM to natively generate watermarked text via a teacher-student framework: applying a watermarked decoding function f_w to an original LLM π_o produces

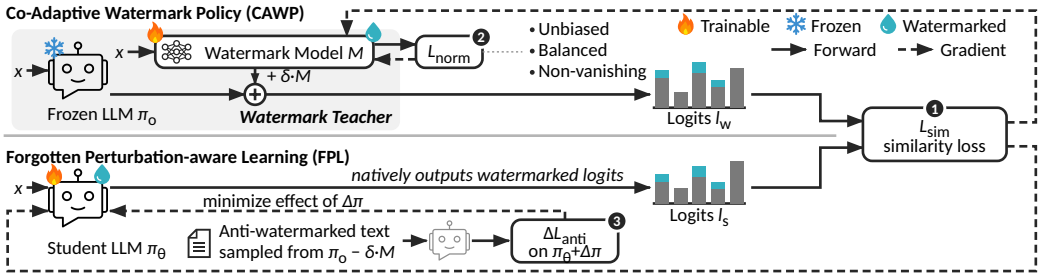


Figure 4: Overview of our proposed method. CAWP (*upper*) jointly trains a watermark model with the student LLM to generate learning-friendly watermark logits. FPL (*bottom*) improves robustness by searching and minimizing the effect of forgotten perturbations that may erase the watermark.

a teacher distribution $f_w(\pi_o(x), \xi)$, and a student LLM π_θ is trained to approximate the teacher's next-token distribution for any input prefix x. After training, the student generates watermarked text under standard decoding. Our goal is to build a precise and robust learning-based watermark that (i) maintains high detection accuracy without degrading generation quality, and (ii) remains robust against user modifications such as fine-tuning, merging, quantization, and pruning. We next discuss the core challenges and present how **OPENMARK** addresses them.

CAWP and FPL address complementary sides of the learnability–robustness trade-off: CAWP co-adapts the student and watermark policy through a shared similarity objective $\mathcal{L}_{\text{sim}}$ (Eq. 3) to establish a learnable signal aligned with detection, and FPL reduces the student's sensitivity to updates that erase this signal. Because both are optimized through $\mathcal{L}_{\text{sim}}$, FPL protects the signal that CAWP teaches the student.

4.1 Co-Adaptive Watermark Policy (CAWP)

Watermark detection for open-source LLMs involves two steps: (1) the LLM user generates text via the watermarked LLM, and (2) the LLM owner uses the predefined watermark pattern to detect the text. In this case, an important consideration is the *consistency* between the watermark pattern learned by the LLM and the predefined one used during detection. To assess this *consistency*, we fine-tune a LLaMA3-8B on text generated by a decoding-based watermarked LLaMA3-8B (i.e., watermark teacher), thereby obtaining a learning-based watermarked LLaMA3-8B (i.e., watermark student). We then measure and compare the green token ratios across them in Figure 5 (left). The results reveal a significant green ratio drift in the student relative to the teacher, which reduces the AUC from 0.99 to 0.84, indicating that the student LLM fails to fully internalize the watermarking pattern of the watermark teacher. To mitigate the inconsistency, we identify two primary optimization directions. The first is to design a learning-friendly watermark pattern, thereby enhancing its inherent learnability for the student LLM. The second is to adapt the detection mechanism itself, using a pattern that aligns more closely with the watermark distribution actually learned by the student LLM, rather than strictly adhering to the original predefined pattern.

Unlike prior work that uses rigid, predefined watermark functions, we introduce Co-Adaptive Watermark Policy (CAWP) as illustrated in Figure 4 (upper). It co-optimizes a trainable watermark policy model with the student LLM, allowing the watermark patterns to adapt to the LLM's learning dynamics and become more learnable. Detection uses the co-optimized patterns instead of the predefined ones, so it matches the watermark the LLM has internalized and avoids generation-detection inconsistency.

The watermark policy consists of a pre-trained Embedding Model E (e.g., BERT [55]) and a trainable Watermark Mapping Model M (e.g., an MLP). Given a sequence of preceding tokens $\{x_{i-n:i-1}\}$ at position i, E generates their embeddings $\{e_{i-n:i-1}\}$. These embeddings are then transformed by M into raw watermark logits over the vocabulary, i.e., $M(E(x_{i-n:i-1}))$. The final watermark logits are obtained by scaling with a strength coefficient δ, yielding $\delta \cdot M(E(x_{i-n:i-1}))$, which are added to the next-token logits before sampling. Given a dataset of texts $\mathcal{D}$, the training objective is to minimize the mean KL divergence between teacher and student next token distributions on all

prefixes in $\mathcal{D}$, shown as ❶ in Figure 4:

$$\begin{aligned}\mathcal{L}_{\text{sim}} &= \tfrac{1}{|\mathcal{D}|}\sum\nolimits_{x,i} \mathrm{KL}\big[P_{\text{tea}}^{(i)} \,\|\, P_{\text{stu}}^{(i)}\big], \quad P_{\text{stu}}^{(i)} = \pi_\theta(\cdot|x_{\le i}),\\ P_{\text{tea}}^{(i)} &= P\big(\log \pi_o(\cdot|x_{\le i}) + \delta \cdot M(E(x_{i-n:i-1}))\big).\end{aligned} \tag{3}$$

Here, n is the prefix context length (in practice, a window of preceding words, e.g., 10 words, which may span a variable number of tokens) and $P(\cdot)$ converts the logits to probabilities. Both the teacher LLM π_o and the embedding model E are frozen; only the student model π_θ and mapping model M are optimized. The student LLM π_θ is optimized by minimizing $\mathcal{L}_{\text{sim}}$; however, naively minimizing $\mathcal{L}_{\text{sim}}$ for M admits degenerate policies. Without further constraints, the joint minimum is dominated by trivial or distribution-distorting solutions, most notably $M \equiv 0$, which erases the watermark. We therefore regularize M with three corresponding requirements, combined into $\mathcal{L}_{\text{norm}}$ and shown as ❷ in Figure 4: (i) *Unbiased Token Preference*: the watermark logits should not exhibit persistent positive or negative bias toward specific tokens. (ii) *Balanced Watermark Logits*: the logits should have zero mean across the vocabulary, ensuring symmetric perturbation that makes approximately half the tokens more likely and the other half less likely. (iii) *Non-Vanishing Watermark Logits*: each watermark logit is encouraged to maintain a minimum absolute magnitude, regularized toward a target value ϵ, preventing degenerate solutions where the watermark logits vanish.

$$\mathcal{L}_{\text{norm}} = \overbrace{\sum\nolimits_j \big|\tfrac{1}{N}\sum\nolimits_i M(e_i)^{(j)}\big|}^{\text{Unbiased Token Preference}} + \overbrace{\sum\nolimits_i \big|\tfrac{1}{|\mathcal{V}|}\sum\nolimits_j M(e_i)^{(j)}\big|}^{\text{Balanced Watermark Logits}} + \overbrace{\lambda_1 \sum\nolimits_{i,j} \max\big(0, \epsilon - |M(e_i)^{(j)}|\big)}^{\text{Non-Vanishing Watermark Logits}} \tag{4}$$

$M(e_i)^{(j)}$ denotes the watermark logit for the j-th token in the vocabulary (of size $|\mathcal{V}|$) corresponding to the i-th input embedding e_i, with N total input samples. The index i sums over all samples, and j sums over all vocabulary tokens. To adjust the green ratio, we can replace the original $M(e_i)^{(j)}$ inside the *Unbiased Token Preference* and *Balanced Watermark Logits* summations with the asymmetric weighted term. So that, for instance, to achieve a green token ratio of 0.25, we can assign a weight 3 to positive logits and 1 to negative logits; the zero-sum constraint ($3 \times N_{pos} - 1 \times N_{neg} \approx 0$) then compels the model to generate three times as many negative tokens as positive ones to maintain equilibrium. This weighted form lets **OPENMARK** specify the target green-token ratio without reverting to a fixed hash table, so the learned policy can still adapt its watermark pattern to the student LLM. More technical details can be found in Appendix B.1.

The final training loss for the watermark mapping model M is the weighted sum of the similarity loss and the normalization loss, given by:

$$\mathcal{L}_M = \mathcal{L}_{\text{sim}} + \lambda_2 \mathcal{L}_{\text{norm}} \tag{5}$$

Upon training, the watermarked student LLM π_θ is released, and the co-optimized watermark mapping model M is retained for detection. For a given text, the LLM owner computes the watermark logit at each position i by first embedding the n-gram prefix with E, then transforming it via the mapping model M to obtain the watermark logit for the actual next token x_i. The detection score is the average watermark logit across the sequence, given by $z = {}^1\!/_N \sum_{i=1}^{N} M\big(E(x_{i-n:i-1})\big)^{(x_i)}$. Because watermarked text is sampled from a teacher whose logits are shifted by $\delta \cdot M(E(\cdot))$, tokens with higher M values are drawn more frequently; the average M value at the realized tokens, i.e., z, is therefore systematically larger for watermarked text than for non-watermarked text. We declare a text watermarked when $z > \tau_{\text{C4}}$, where τ_{C4} is calibrated once as the empirical 99th-percentile score on an independent set of 200-token human-C4 texts, disjoint from evaluation. The detector and threshold are then frozen; Appendix D.13 reports realized FPR and its dependence on text length.

While some prior work also employs neural networks to generate watermark logits, they target different goals. For example, [56], [57] and [58] design semantic-invariant or public-verifiable watermark models to improve robustness or enable third-party verification. Existing approaches are designed for closed-source LLMs and do not consider the learnability of the watermark by the model itself—their watermark models are trained independently from the LLM. By contrast, CAWP focuses on generating *learning-friendly* watermark logits by jointly optimizing the watermark model with the student LLM being watermarked. Additionally, such joint training ensures a bidirectional alignment: the watermark pattern used for detection adapts toward the student LLM's learned representation, while the student LLM is simultaneously guided to internalize patterns consistent with detection. This mutual convergence narrows the gap between the student and teacher distributions in Figure 5 (left).

4.2 Forgotten Perturbation-aware Learning (FPL)

The vulnerability of watermarked LLMs to user modifications arises primarily from *forgotten perturbation*. Specifically, when a user modifies a watermarked LLM π_θ into π'_θ, the weight drift $\Delta\pi_\theta = \pi'_\theta - \pi_\theta$ may remove the learned watermark. We term the drift that erases the watermark as *forgotten perturbation*, and our goal is to minimize its effect during the watermark learning stage.

To understand its root cause, we investigate fine-tuning as a representative user modification, i.e., $\pi_{t+1} = \pi_t - \eta\, g(\pi_t)$, where $g(\pi_t)$ is the gradient on user data driving weight drift. Figure 5 (right) shows that fine-tuning a watermarked LLaMA3-8B on raw texts (a mixture of green/red tokens) collapses detectability, whereas fine-tuning on watermarked texts (green-token dominant) preserves it. This indicates that gradients arising from red tokens are the primary cause of *forgotten perturbation*. To further validate this, we fine-tune on *anti-watermarked* texts, generated with inverted watermark logits to make texts red-token dominant. As shown in Figure 5 (right), fine-tuning on these anti-watermarked texts results in a substantial drop in detectability. These results confirm that red-token-driven *forgotten perturbations* are the dominant factor in watermark forgetting, and mitigating their impact should be a key objective.

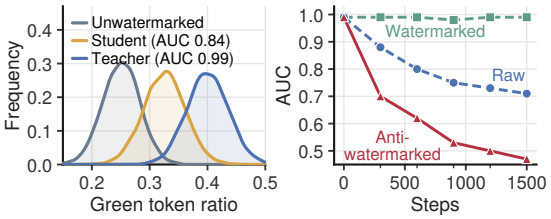


Figure 5: *(Left)* Green token ratios for unwatermarked, watermark student, and watermark teacher (KGW with green ratio $= 0.25$, $n = 1$, $\delta = 1$). *(Right)* AUC during fine-tuning on raw, watermarked, and anti-watermarked texts.

To achieve this, we propose Forgotten Perturbation-aware Learning (FPL), which targets the worst-case erasure direction, i.e., the red-token gradient direction identified above, rather than building defenses for each user modification type, as illustrated in Figure 4 (bottom). We use anti-watermarked data as a proxy whose gradient $\nabla\mathcal{L}_{\text{anti}}$ aligns with this direction, and apply a SAM-style sharpness penalty along it, so that $\mathcal{L}_{\text{anti}}$ stays locally flat and user updates along this subspace cause minimal watermark loss. The resulting training objective of the student LLM is:

$$\arg\min_{\boldsymbol{\pi}_\theta} \mathcal{L}_{\text{sim}}(\boldsymbol{\pi}_\theta) + \beta(\mathcal{L}_{\text{anti}}(\boldsymbol{\pi}_\theta) - \mathcal{L}_{\text{anti}}(\boldsymbol{\pi}_\theta - \alpha\frac{\nabla\mathcal{L}_{\text{anti}}(\boldsymbol{\pi}_\theta)}{\|\nabla\mathcal{L}_{\text{anti}}(\boldsymbol{\pi}_\theta)\|})) \tag{6}$$

Here, $\mathcal{L}_{\text{sim}}$ is the watermark learning loss over watermarked texts (Eq. 3), and $\mathcal{L}_{\text{anti}}$ is its analog evaluated on anti-watermarked texts (a KL divergence between the inverted teacher and the student). We denote the bracketed term by $\Delta\mathcal{L}_{\text{anti}}$, shown as ❸ in Figure 4; it is the drop of $\mathcal{L}_{\text{anti}}$ under the forgotten perturbation $\Delta\pi = -\alpha\nabla\mathcal{L}_{\text{anti}}/\|\nabla\mathcal{L}_{\text{anti}}\|$. Minimizing it keeps $\mathcal{L}_{\text{anti}}$ flat along this direction, so that the watermark is not forgotten. The hyperparameter β controls the trade-off between watermark strength and robustness against red-token forgetting, while α is the perturbation step size. Optimizing Equation 6 involves second-order derivatives; a first-order approximation reduces each step to three forward/backward passes (Appendix A.4).

4.3 Training Procedure

Two modules are frozen throughout: the teacher LLM π_o and the embedding model E; two are trainable: the student LLM π_θ and the watermark mapping model M. At each iteration, a minibatch updates both: π_θ minimizes the FPL objective (Eq. 6, which combines $\mathcal{L}_{\text{sim}}$ with the forgotten-perturbation regularizer), while M minimizes $\mathcal{L}_M = \mathcal{L}_{\text{sim}} + \lambda_2\mathcal{L}_{\text{norm}}$ (Eq. 5). In Figure 4, M is thus updated by ❶+❷ and π_θ by ❶+❸.

Appendix A gives proxy-loss analyses of CAWP (convergence, detectability) and FPL (robustness, overhead); they motivate the design, and detection performance is evaluated empirically in Section 5.

5 Evaluation

We evaluate **OpenMark** on text generation, then assess the range of model and output types supported by **OpenMarkX**.

5.1 Experimental settings

Models. We perform experiments on three open-source LLMs: LLaMA3-8B, Phi2-2.7B, and LLaMA3.2-3B, which cover a range of model sizes. The watermark mapping model M is a lightweight MLP: a linear projection $\mathbb{R}^{1024} \to \mathbb{R}^{500}$, two ReLU-based residual blocks, and a final projection to the vocabulary dimension with $\tanh$ to constrain logits to $[-1, 1]$.

Implementation Details. The embedding model and all hyperparameters are listed in Appendix C.2. All main-text **OpenMark** configurations use a prefix context length of $n = 10$ words on every evaluated model, and generation uses sampling with temperature 1.0 by default (other decoding settings are evaluated in Appendix D.15). Appendix D.14 reports 95% bootstrap confidence intervals and three-seed results. **OpenMark** adds negligible training cost: the watermark mapping model has 1.16M parameters, and FPL's extra passes are offset by faster convergence (Appendix D.1).

Baseline Watermarking Methods. Our experiments use three baseline watermarking methods, all via KL-based distillation from a decoding-based teacher model. The first is Gu et al.-KGW: following [24], we distill KGW with fixed $\gamma = 0.25$ and evaluate three (k, δ) configurations: $(1, 2)$, $(0, 1)$, and $(1, 1)$. The second is Gu et al.-KTH, which distills the exponential decoding watermark of [20]. The third is Gloaguen et al.-KGW, which embeds KGW watermarks into "stable" parameters identified via contrastive task vector analysis before and after fine-tuning on raw data. Training configurations and device usage are in Appendix C.3.

Metrics. We evaluate open-source watermarking methods along three dimensions: detectability, text quality, and robustness. For detectability, following [24], each watermarked model generates 5,000 samples from 50-token C4 prefixes [59] with 200-token continuations. Text quality is measured by median perplexity (PPL) using a LLaMA-2-13B model following [24], together with downstream reasoning accuracy on HellaSwag, ARC-Challenge, AGIEval-Math, and GSM8K to capture utility beyond distributional fluency [17]. To assess robustness, we apply four model modifications simulating real-world user behavior; settings are in Appendix C.3.

Detector calibration. We set each detector's threshold to the 99th percentile of scores on a separate calibration set of 200-token human-written C4 texts. We freeze this threshold for evaluation. The observed false-positive rates are 1.06% on held-out human C4 text, 1.28% on unwatermarked outputs from the same base model, and 0.82% on Qwen3-4B outputs (Appendix D.13).

5.2 Results

Detectability and generation quality. Figure 6 compares **OpenMark** with the methods of [24] and [27] as watermark strength varies (settings in Appendix C.1). **OpenMark** reaches 0.92 TPR at 1% FPR and reduces PPL by 20.5% on average across three LLMs relative to Gu et al.-KGW, with higher TPR at 1% FPR on each model (Appendix D.14). Figure 10, Table 9, and Figure 9 give AUC, results at 0.1% and 10% FPR, and ROC curves.

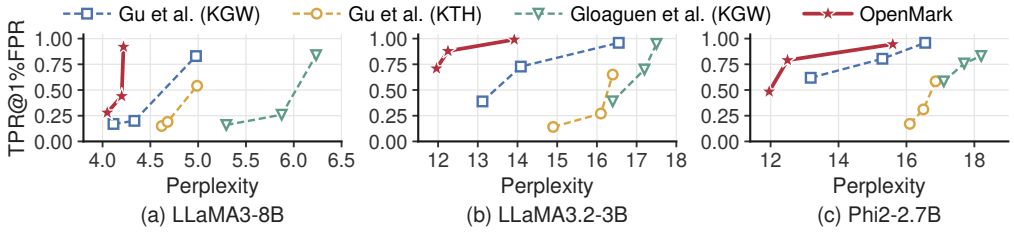


Figure 6: Effectiveness comparison of open-source LLM text watermarks using detection TPR at 1% FPR (TPR@1) and median PPL on three LLMs. Better detectability and generation quality correspond to higher TPR@1 and lower PPL, i.e., lines closer to the upper left. Original models' PPL: LLaMA3-8B = 3.6, LLaMA3.2-3B = 10.9, Phi2-2.7B = 11.3.

Downstream Task Preservation. Beyond PPL, we evaluate whether watermarking degrades reasoning ability. Table 1 reports accuracy on four benchmarks for base LLaMA-3-8B, the KGW-watermarked variant [24], and **OpenMark**. Across the four tasks, **OpenMark** accuracy drops by at most 0.43 percentage points relative

Table 1: Downstream task accuracy of LLaMA-3-8B before and after watermarking. **OpenMark** preserves capability.

Model	Hella.	ARC-C	AGI-M	GSM8K
Base	0.7607	0.5708	0.2963	0.7583
Gu et al.-KGW	0.7583	0.5683	0.2877	0.7582
OpenMark	0.7594	0.5705	0.2920	0.7582

Table 2: TPR at 5%, 1% FPR and PPL of different watermark methods of LLaMA-3-8B under model modifications. The perplexity of the original LLaMA-3-8B without watermarking is 3.6. TPR@1% is the primary robustness metric; selected bootstrap intervals and separate three-seed results appear in Appendix D.14. We lightly highlight TPR@5% values with green (≥ 0.8) and yellow (0.5–0.8) for readability.

Category	Method	Config	[24]-KTH			[24]-KGW			[27]-KGW			OpenMark		
			TPR@5	**TPR@1**	PPL	TPR@5	**TPR@1**	PPL	TPR@5	**TPR@1**	PPL	TPR@5	**TPR@1**	PPL
Unaltered			0.78	0.54	5.0	0.96	0.83	4.9	0.94	0.84	6.2	0.99	0.92	4.2
Quantization	8 bits	GPTQ	0.66	0.42	5.2	0.92	0.79	5.0	0.95	0.84	6.3	0.97	0.86	4.3
		INT8	0.69	0.45	5.1	0.91	0.73	5.0	0.94	0.84	6.4	0.93	0.77	4.4
	4 bits	HQQ	0.54	0.29	6.1	0.95	0.83	5.5	0.91	0.77	7.9	0.98	0.91	4.8
		GPTQ	0.64	0.36	5.5	0.91	0.74	5.1	0.94	0.81	7.3	0.94	0.79	5.1
Pruning	Wanda	$\rho = 0.2$	0.73	0.47	8.6	0.96	0.84	7.5	0.97	0.90	9.9	0.99	0.92	7.0
		$\rho = 0.5$	0.64	0.35	8.4	0.86	0.66	6.9	0.93	0.80	8.9	0.96	0.84	6.2
	SparseGPT	$\rho = 0.2$	0.78	0.55	7.7	0.95	0.82	8.1	0.95	0.84	9.0	0.98	0.91	7.1
		$\rho = 0.5$	0.76	0.50	8.1	0.91	0.74	7.8	0.93	0.78	9.6	0.97	0.89	8.0
Merging	SLERP	$t = 0.1$	0.36	0.15	5.2	0.76	0.53	4.3	0.82	0.59	6.1	0.92	0.78	4.2
		$t = 0.3$	0.21	0.08	5.1	0.65	0.39	4.0	0.62	0.37	5.3	0.83	0.60	4.1
		$t = 0.5$	0.09	0.02	4.8	0.30	0.12	3.8	0.34	0.14	4.6	0.57	0.28	4.0
		$t = 0.7$	0.06	0.02	4.3	0.15	0.05	3.7	0.19	0.06	4.2	0.31	0.12	3.9
		$t = 0.9$	0.06	0.01	4.1	0.11	0.03	3.6	0.11	0.03	4.0	0.17	0.06	3.8
Finetuning	OpenMath Instruct	$s = 300$	0.21	0.08	5.1	0.48	0.20	4.1	0.45	0.22	5.5	0.70	0.43	4.1
		$s = 600$	0.17	0.05	4.9	0.38	0.16	3.7	0.41	0.19	5.2	0.51	0.26	3.9
		$s = 900$	0.09	0.03	4.8	0.30	0.11	3.6	0.31	0.13	5.2	0.46	0.22	3.7
		$s = 1200$	0.09	0.02	4.7	0.25	0.09	3.5	0.25	0.10	5.0	0.40	0.18	3.6
		$s = 1500$	0.02	0.02	4.5	0.23	0.08	3.5	0.22	0.08	5.1	0.37	0.17	3.6
	OpenCode Instruct	$s = 300$	0.27	0.11	7.3	0.63	0.40	6.9	0.65	0.41	7.4	0.71	0.43	7.0
		$s = 600$	0.22	0.08	5.7	0.45	0.24	4.9	0.48	0.23	7.0	0.56	0.31	5.4
		$s = 900$	0.16	0.06	4.8	0.32	0.14	5.0	0.34	0.16	6.8	0.50	0.25	5.1
		$s = 1200$	0.11	0.03	4.7	0.29	0.12	4.3	0.31	0.12	6.3	0.40	0.18	4.8
		$s = 1500$	0.07	0.02	4.6	0.26	0.10	4.2	0.30	0.12	5.9	0.39	0.17	4.0

to the unwatermarked model. The largest drop occurs on AGIEval-Math (0.2963 to 0.2920).

Robustness against Model Modifications. Following [27], we evaluate **OpenMark** under common model modifications, including quantization, pruning, model merging, and fine-tuning [49]. As shown in Table 2, existing methods remain reliable under *quantization* and *pruning*, where parameter shifts are limited and learned weights are only mildly affected. However, under *model merging* and *fine-tuning*, which cause larger and less predictable weight changes, existing methods degrade noticeably. **OpenMark** degrades more slowly than all baselines under both. Merging with the original model at $t = 0.5$ leaves **OpenMark** with TPR@5%/1% FPR of 0.57/0.28, versus at most 0.34/0.14 for baselines. Under OpenMathInstruct fine-tuning, **OpenMark** retains 0.70/0.43 after 300 steps and 0.37/0.17 after 1,500 steps, versus at most 0.48/0.22 and 0.23/0.08 for baselines; **OpenMark** also leads at every step on OpenCodeInstruct. FPL's controlled ablation and mechanism diagnostics are in Appendices D.3 and D.17.

Robustness against Text Paraphrasing. **OpenMark** is more robust to paraphrasing attacks [45, 46] than baselines, achieving an AUC of 0.90 under DIPPER [45], compared to 0.86 for Gu et al.-KGW (See Appendix D.7). Under Qwen3.5-9B paraphrasing, **OpenMark** retains TPR@1% FPR of 0.32 versus 0.25 for Gu et al.-KGW after one round, and 0.17 versus 0.12 after three rounds.

Short texts. At 100 tokens on Granite-4.2-3B, **OpenMark** reaches 0.935 TPR at 1% FPR using the text alone and a length-specific threshold. Appendix D.13, Table 18, compares both methods across all five lengths.

Cross-model attribution. Can an owner distinguish its model's outputs from those of another model carrying an **OpenMark** watermark? We independently watermark three models and apply each detector to every generator's outputs (Table 3). At a nominal 1% FPR, the detectors flag 90.9%–92.2% of outputs from their own models and 0.4%–1.3% from the other two.

Generalization to OOD Prompts. **OpenMark** trained on C4 keeps the best detectability-quality trade-off on Wikipedia prompts without retraining, and the same pipeline generalizes to GPT-J-6B (Appendix D.2).

Table 3: Cross-model attribution at a nominal 1% FPR. Each cell gives the percentage of outputs from the column model flagged by the row detector. Bold entries pair each detector with its own model.

	Generator		
Detector	LLaMA-3-8B	GPT-J-6B	Phi-2-2.7B
$M_{\text{LLaMA-3-8B}}$	**92.2%**	1.3%	0.8%
$M_{\text{GPT-J-6B}}$	0.9%	**90.9%**	1.1%
$M_{\text{Phi-2-2.7B}}$	0.4%	0.7%	**91.2%**

Table 4: Watermarking demonstrations supported by **OPENMARKX** across model and output types, using compatible methods and adapters. TPR at 1% FPR; quality drop is the relative change of each model's own metric from the unwatermarked to the watermarked model.

Output	Model	Policy acts on	TPR@1%	Quality drop
Text from image	MedGemma-1.5-4B	next-token logits	0.86	0.0% (Acc.)
Agent answers	Llama-3.1-8B (ReAct)	next-token logits	0.82	8.4% (Acc.)
Image	Taming (ImageNet)	image-codebook tokens	0.90	4.6% (FID)
Physics field	GeoPT (aircraft)	keyed ripple on the field	1.00	0.6% (Rel. L_2)

Other Experiments. Appendix D.15 varies the decoder, and Appendices D.8 and D.3 vary fine-tuning duration and FPL strength. Replacing the learned policy with a fixed neural policy [56] reduces TPR at 1% FPR from 0.92 to 0.86 and raises PPL from 4.2 to 4.6 (Appendix D.16). Appendices D.11, D.9, D.10, and D.12 examine the embedding model, attribution, and distillation data scale.

Platform coverage. We examine provenance beyond standalone text in the context of model reuse and multimodal systems [60–63]. Table 4 uses a frozen-policy **OPENMARK** variant for medical answers [64, 65] and ReAct final answers [66, 67], WMAR for generated images [54, 68], and Ripple for physics fields [69]. Across these four configurations TPR ranges from 0.82 to 1.00 at 1% FPR.

Watermarking agents result. For the Llama-3.1-8B ReAct agent, the watermark policy acts on next-token logits. At a 1% false positive rate, the detector identifies 82% of watermarked final answers (Table 4). Task accuracy drops by 8.4% relative to the unwatermarked agent, the largest relative quality drop in the table. This evaluation tests the final answer only; it does not test the agent's intermediate actions.

6 Conclusion

OPENMARKX lets a model owner select a compatible watermark and evaluate its signal after editing the released weights or outputs. Across three text models, **OPENMARK** reduces perplexity by 20.5% on average relative to Gu et al.-KGW and retains 0.57 TPR at 5% FPR after a 0.5-ratio merge, versus at most 0.34 for the baselines. The platform also supports evaluation on agent answers, medical answers, images, and physical fields.

Acknowledgements

This work was supported in part by the National Science Foundation under Award No. 2530786. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of the National Science Foundation.

References

[1] Jiaqi Xue, Qian Lou, Jiarong Xing, and Heng Huang. R2-router: A new paradigm for LLM routing with reasoning. In Tong Zhang, Miroslav Dudik, Martin Jaggi, Alekh Agarwal, Sharon Li, Dale Schuurmans, Jerry Zhu, Felix Berkenkamp, Hanze Dong, and Alberto Bietti, editors, *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, pages 143255–143268. PMLR, 06–11 Jul 2026. URL `https://proceedings.mlr.press/v306/xue26h.html`.

[2] Yifei Zhao, Qian Lou, and Mengxin Zheng. Realm: A unified red-teaming benchmark for physical-world vlms. *arXiv preprint arXiv:2606.23892*, 2026.

[3] Ahasan Kabir, Jiaqi Xue, Mengxin Zheng, and Qian Lou. Hw-router: Hardware-aware routing for scalable multi-llm serving. In *Proceedings of the 63rd Design Automation Conference (DAC)*, 2026.

[4] Jiaqi Xue, Mengxin Zheng, Heng Huang, and Qian Lou. Chain-of-route: State-aware LLM routing for multi-turn conversations. In *The Fortieth Annual Conference on Neural Information Processing Systems*, 2026. URL https://openreview.net/forum?id=CPCK7uqzh3.

[5] Jiaqi Xue, Yanjun Wang, Xiangci Li, Lingbo Mo, Aritra Sengupta, Shweta Garg, Murali Krishna Ramanathan, and Myeongsoo Kim. Aecp: Artifact-exclusive communication protocol for multi-agent code generation. *arXiv preprint arXiv:2610.06481*, 2026.

[6] Mayank Kumar, Jiaqi Xue, Mengxin Zheng, and Qian Lou. FHE-Coder: Benchmarking secure agentic code generation for fully homomorphic encryption. In *International Conference on Learning Representations*, 2026.

[7] Ahasan Kabir, Jiaqi Xue, Mengxin Zheng, and Qian Lou. INFRAMIND: Infrastructure-aware multi-agent orchestration. *arXiv preprint arXiv:2606.11440*, 2026.

[8] Rishi Bommasani, Drew A Hudson, Ehsan Adeli, Russ Altman, Simran Arora, Sydney von Arx, Michael S Bernstein, Jeannette Bohg, Antoine Bosselut, Emma Brunskill, et al. On the opportunities and risks of foundation models. *arXiv preprint arXiv:2108.07258*, 2021.

[9] E Union. Proposal for a regulation of the european parliament and of the council laying down harmonised rules on artificial intelligence (artificial intelligence act) and amending certain union legislative acts. *COM/2021/206final*, 2021.

[10] Jiaqi Xue, Mengxin Zheng, Yebowen Hu, Fei Liu, Xun Chen, and Qian Lou. BadRAG: Identifying vulnerabilities in retrieval augmented generation of large language models. *arXiv preprint arXiv:2406.00083*, 2024.

[11] Jiaqi Xue, Mengxin Zheng, Ting Hua, Yilin Shen, Yepeng Liu, Ladislau Bölöni, and Qian Lou. Trojllm: A black-box trojan prompt attack on large language models. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36, pages 65665–65677. Curran Associates, Inc., 2023. doi: 10.52202/075280-2866. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/cf04d01a0e76f8b13095349d9caca033-Paper-Conference.pdf.

[12] Jiaqi Xue, Qian Lou, and Mengxin Zheng. Badfair: Backdoored fairness attacks with group-conditioned triggers. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 8257–8270, 2024.

[13] Mengxin Zheng, Jiaqi Xue, Xun Chen, Yanshan Wang, Qian Lou, and Lei Jiang. TrojFSP: Trojan insertion in few-shot prompt tuning. In Kevin Duh, Helena Gomez, and Steven Bethard, editors, *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 1141–1151, Mexico City, Mexico, June 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.naacl-long.64. URL https://aclanthology.org/2024.naacl-long.64/.

[14] Rochana Prih Hastuti, Rian Adam Rajagede, Mansour Al Ghanim, Mengxin Zheng, and Qian Lou. Factuality beyond coherence: Evaluating LLM watermarking methods for medical texts. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng, editors, *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 15129–15147, Suzhou, China, November 2025. Association for Computational Linguistics. ISBN 979-8-89176-335-7. doi: 10.18653/v1/2025.findings-emnlp.818. URL https://aclanthology.org/2025.findings-emnlp.818/.

[15] Mansour Al Ghanim, Jiaqi Xue, Rochana Prih Hastuti, Mengxin Zheng, Yan Solihin, and Qian Lou. Evaluating the robustness and accuracy of text watermarking under real-world cross-lingual manipulations. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng, editors, *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 7396–7416, Suzhou, China, November 2025. Association for Computational Linguistics. ISBN 979-8-89176-335-7. doi: 10.18653/v1/2025.findings-emnlp.390. URL https://aclanthology.org/2025.findings-emnlp.390/.

[16] Qian Lou, Xin Liang, Jiaqi Xue, Yancheng Zhang, Rui Xie, and Mengxin Zheng. Cr-utp: Certified robustness against universal text perturbations on large language models. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 9863–9875, 2024.

[17] Rochana Prih Hastuti, Rian Adam Rajagede, Mengxin Zheng, and Qian Lou. Evaluating LLM watermarking for clinical EHR: Detectability, style shifts, and helpfulness. In *Proceedings of the 2026 Conference on Empirical Methods in Natural Language Processing*, pages 58616–58639, 2026.

[18] Yifei Zhao, Qian Lou, and Mengxin Zheng. Sif: Semantically in-distribution fingerprints for large vision-language models. *ArXiv*, abs/2604.17041, 2026. URL https://api.semanticscholar.org/CorpusID:287634725.

[19] Ruisi Zhang, Yifei Zhao, Neusha Javidnia, Mengxin Zheng, and Farinaz Koushanfar. Attestllm: Efficient attestation framework for billion-scale on-device llms. In *Design Automation Conference (DAC)*, 2026.

[20] Rohith Kuditipudi, John Thickstun, Tatsunori Hashimoto, and Percy Liang. Robust distortion-free watermarks for language models. *Transactions on Machine Learning Research*, 2024. ISSN 2835-8856. URL https://openreview.net/forum?id=FpaCL1MO2C.

[21] John Kirchenbauer, Jonas Geiping, Yuxin Wen, Jonathan Katz, Ian Miers, and Tom Goldstein. A watermark for large language models. In *International Conference on Machine Learning*, pages 17061–17084. PMLR, 2023.

[22] Zhengmian Hu, Lichang Chen, Xidong Wu, Yihan Wu, Hongyang Zhang, and Heng Huang. Unbiased watermark for large language models. *arXiv preprint arXiv:2310.10669*, 2023.

[23] Miranda Christ, Sam Gunn, Tal Malkin, and Mariana Raykova. Provably robust watermarks for open-source language models. *arXiv preprint arXiv:2410.18861*, 2024.

[24] Chenchen Gu, Xiang Lisa Li, Percy Liang, and Tatsunori Hashimoto. On the learnability of watermarks for language models. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=9k0krNzvlV.

[25] Tom Sander, Pierre Fernandez, Alain Durmus, Matthijs Douze, and Teddy Furon. Watermarking makes language models radioactive. *Advances in Neural Information Processing Systems*, 37:21079–21113, 2024.

[26] Zhengyue Zhao, Xiaogeng Liu, Somesh Jha, Patrick McDaniel, Bo Li, and Chaowei Xiao. Can watermarks be used to detect llm ip infringement for free? In *The Thirteenth International Conference on Learning Representations*, 2025.

[27] Thibaud Gloaguen, Nikola Jovanović, Robin Staab, and Martin Vechev. Towards watermarking of open-source llms. *arXiv preprint arXiv:2502.10525*, 2025.

[28] Xuandong Zhao, Prabhanjan Vijendra Ananth, Lei Li, and Yu-Xiang Wang. Provable robust watermarking for AI-generated text. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=SsmT8aO45L.

[29] John Kirchenbauer, Jonas Geiping, Yuxin Wen, Manli Shu, Khalid Saifullah, Kezhi Kong, Kasun Fernando, Aniruddha Saha, Micah Goldblum, and Tom Goldstein. On the reliability of watermarks for large language models. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=DEJIDCmWOz.

[30] Xiaojun Xu, Yuanshun Yao, and Yang Liu. Learning to watermark LLM-generated text via reinforcement learning. In *ICLR 2025 Workshop on GenAI Watermarking*, 2025. URL https://openreview.net/forum?id=dTlCMiEOdk.

[31] Adam Block, Ayush Sekhari, and Alexander Rakhlin. Gaussmark: A practical approach for structural watermarking of language models. *arXiv preprint arXiv:2501.13941*, 2025.

[32] Kaibo Huang, Jin Tan, Yukun Wei, Wanling Li, Zipei Zhang, Hui Tian, Zhongliang Yang, and Linna Zhou. AgentMark: Utility-preserving behavioral watermarking for agents. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 12581–12603. Association for Computational Linguistics, 2026. doi: 10.18653/v1/2026.acl-long.573. URL https://aclanthology.org/2026.acl-long.573/.

[33] Wenlong Meng, Chen Gong, Terry Yue Zhuo, Fan Zhang, Kecen Li, Zheng Liu, Zhou Yang, Chengkun Wei, and Wenzhi Chen. Watermarking LLM agent trajectories. *arXiv preprint arXiv:2602.18700*, 2026. doi: 10.48550/arXiv.2602.18700. URL https://arxiv.org/abs/2602.18700.

[34] Hyeseon An, Shinwoo Park, Dongsu Kim, and Yo-Sub Han. Sequential behavioral watermarking for LLM agents. *arXiv preprint arXiv:2605.11036*, 2026. doi: 10.48550/arXiv.2605.11036. URL https://arxiv.org/abs/2605.11036.

[35] Yancheng Zhang, Jiaqi Xue, Mengxin Zheng, Mimi Xie, Mingzhe Zhang, Lei Jiang, and Qian Lou. CipherPrune: Efficient and scalable private transformer inference. In *International Conference on Learning Representations*, 2025.

[36] Muhammad Husni Santriaji, Jiaqi Xue, Qian Lou, and Yan Solihin. DataSeal: Ensuring the verifiability of private computation on encrypted data. In *IEEE Symposium on Security and Privacy*, 2025.

[37] Jiaqi Xue, Mengxin Zheng, and Qian Lou. RobPI: Robust private inference against malicious client. In *IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*, 2026.

[38] Jiaqi Xue, Mayank Kumar, Yuzhang Shang, Shangqian Gao, Rui Ning, Mengxin Zheng, Xiaoqian Jiang, and Qian Lou. Dictpfl: Efficient and private federated learning on encrypted gradients. *Advances in Neural Information Processing Systems*, 38:119806–119829, 2026.

[39] Jiaqi Xue, Yancheng Zhang, Yanshan Wang, Xueqiang Wang, Hao Zheng, and Qian Lou. CryptoTrain: Fast secure training on encrypted dataset. In *Workshop on Large AI Systems and Models with Privacy and Safety Analysis (LAMPS) at ACM CCS*, 2024.

[40] Ardhi Wiratama Baskara Yudha, Jiaqi Xue, Qian Lou, Huiyang Zhou, and Yan Solihin. BoostCom: Towards efficient universal fully homomorphic encryption by boosting the word-wise comparisons. In *International Conference on Parallel Architectures and Compilation Techniques (PACT)*, 2024.

[41] Jiaqi Xue, Xin Xin, Wei Zhang, Mengxin Zheng, Qianqian Song, Minxuan Zhou, Yushun Dong, Dongjie Wang, Xun Chen, Jiafeng Xie, Liqiang Wang, David Mohaisen, Hongyi Wu, and Qian Lou. SoK: Can fully homomorphic encryption support general AI computation? a functional and cost analysis. *Proceedings on Privacy Enhancing Technologies (PoPETs)*, 2026.

[42] Jiaqi Xue, Yifei Zhao, Mengxin Zheng, Fan Yao, Yan Solihin, and Qian Lou. Securing transformer-based AI execution via unified TEEs and crypto-protected accelerators. *arXiv preprint arXiv:2507.03278*, 2025.

[43] Qian Lou, Muhammad Santriaji, Ardhi Wiratama Baskara Yudha, Jiaqi Xue, and Yan Solihin. vFHE: Verifiable fully homomorphic encryption with blind hash. *arXiv preprint arXiv:2303.08886*, 2023.

[44] Jiaqi Xue, Lei Xu, Lin Chen, Weidong Shi, Kaidi Xu, and Qian Lou. Audit and improve robustness of private neural networks on encrypted data. *arXiv preprint arXiv:2209.09996*, 2022.

[45] Kalpesh Krishna, Yixiao Song, Marzena Karpinska, John Wieting, and Mohit Iyyer. Paraphrasing evades detectors of ai-generated text, but retrieval is an effective defense. *Advances in Neural Information Processing Systems*, 36:27469–27500, 2023.

[46] Mansour Al Ghanim, Saleh Almohaimeed, Mengxin Zheng, Yan Solihin, and Qian Lou. Jailbreaking LLMs with Arabic transliteration and Arabizi. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 18584–18600, 2024.

[47] Qian Lou, Yepeng Liu, and Bo Feng. TrojText: Test-time invisible textual trojan insertion. In *International Conference on Learning Representations*, 2023.

[48] Mansour Al Ghanim, Muhammad Santriaji, Qian Lou, and Yan Solihin. TrojBits: A hardware aware inference-time attack on transformer-based language models. In *European Conference on Artificial Intelligence*, pages 60–68, 2023.

[49] Kunbei Cai, Zhenkai Zhang, Qian Lou, and Fan Yao. WBP: Training-time backdoor attacks through hardware-based weight bit poisoning. In *European Conference on Computer Vision*, pages 179–197, 2024.

[50] Mengxin Zheng, Jiaqi Xue, Zihao Wang, Xun Chen, Qian Lou, Lei Jiang, and Xiaofeng Wang. SSL-Cleanse: Trojan detection and mitigation in self-supervised learning. In *European Conference on Computer Vision*, 2024.

[51] Mengxin Zheng, Jiaqi Xue, Yi Sheng, Lei Yang, Qian Lou, and Lei Jiang. TrojFair: Trojan fairness attacks. In *Workshop on Large AI Systems and Models with Privacy and Safety Analysis (LAMPS) at ACM CCS*, 2024.

[52] Jiaqi Xue and Qian Lou. ESTAS: Effective and stable trojan attacks in self-supervised encoders with one target unlabelled sample. *arXiv preprint arXiv:2211.10908*, 2022.

[53] Leyi Pan, Aiwei Liu, Zhiwei He, Zitian Gao, Xuandong Zhao, Yijian Lu, Binglin Zhou, Shuliang Liu, Xuming Hu, Lijie Wen, Irwin King, and Philip S. Yu. MarkLLM: An open-source toolkit for LLM watermarking. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 61–71, 2024.

[54] Nikola Jovanović, Ismail Labiad, Tomáš Souček, Martin Vechev, and Pierre Fernandez. Watermarking autoregressive image generation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[55] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, pages 4171–4186, 2019.

[56] Aiwei Liu, Leyi Pan, Xuming Hu, Shiao Meng, and Lijie Wen. A semantic invariant robust watermark for large language models. In *The Twelfth International Conference on Learning Representations*, 2024. URL `https://openreview.net/forum?id=6p8lpe4MNf`.

[57] Jie Ren, Han Xu, Yiding Liu, Yingqian Cui, Shuaiqiang Wang, Dawei Yin, and Jiliang Tang. A robust semantics-based watermark for large language model against paraphrasing. In *Findings of the Association for Computational Linguistics: NAACL 2024*, 2024.

[58] Aiwei Liu, Leyi Pan, Xuming Hu, Shuang Li, Lijie Wen, Irwin King, and Philip S. Yu. An unforgeable publicly verifiable watermark for large language models. In *The Twelfth International Conference on Learning Representations*, 2024. URL `https://openreview.net/forum?id=gMLQwKDY3N`.

[59] Jesse Dodge, Maarten Sap, Ana Marasović, William Agnew, Gabriel Ilharco, Dirk Groeneveld, Margaret Mitchell, and Matt Gardner. Documenting large webtext corpora: A case study on the colossal clean crawled corpus. *arXiv preprint arXiv:2104.08758*, 2021.

[60] Kunyang Li, Mubarak Shah, and Yuzhang Shang. PackCache: A training-free acceleration method for unified autoregressive video generation via compact KV-cache. *arXiv preprint arXiv:2601.04359*, 2026.

[61] Kunyang Li, Mubarak Shah, and Yuzhang Shang. Attend locally, remember linearly: Linear attention as cross-frame memory for autoregressive video diffusion. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2026.

[62] Abdul Mohaimen Al Radi, Kunyang Li, Yuzhang Shang, Mubarak Shah, and Yu Tian. Keepsake: Selective spatial memory for long-horizon video generation. *arXiv preprint arXiv:2610.06588*, 2026.

[63] Abdul Mohaimen Al Radi, Kunyang Li, Yuzhang Shang, Mubarak Shah, and Yu Tian. Aero-World: Action-conditioned aerial video generation from inertial controls. *arXiv preprint arXiv:2605.19728*, 2026.

[64] Andrew Sellergren et al. MedGemma technical report. *arXiv preprint arXiv:2507.05201*, 2025.

[65] Kunyang Li, Hai Nguyen, Joshua Lowe, Chenguang Zhao, Peace C. Madueme, Mehdi Hedjazi Moghari, Mubarak Shah, Pegah Khosravi, and Yuzhang Shang. CineMR: Tool-integrated vision-language reasoning for quantitative cardiac MRI assessment. *arXiv preprint arXiv:2610.01166*, 2026.

[66] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

[67] AI@Meta. Llama 3 model card. 2024. URL `https://github.com/meta-llama/llama3/blob/main/MODEL_CARD.md`.

[68] Patrick Esser, Robin Rombach, and Björn Ommer. Taming transformers for high-resolution image synthesis. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 12873–12883, 2021.

[69] Haixu Wu, Minghao Guo, Zongyi Li, Zhiyang Dou, Mingsheng Long, Kaiming He, and Wojciech Matusik. GeoPT: Scaling physics simulation via lifted geometric pre-training. *arXiv preprint arXiv:2602.20399*, 2026.

[70] Peter L Bartlett and Shahar Mendelson. Rademacher and gaussian complexities: Risk bounds and structural results. *Journal of Machine Learning Research*, 2002.

[71] Behnam Neyshabur, Ryota Tomioka, and Nati Srebro. Norm-based capacity control in neural networks. In *COLT*, 2015.

[72] Peter L Bartlett, Dylan J Foster, and Matus Telgarsky. Spectrally-normalized margin bounds for neural networks. In *NeurIPS*, 2017.

[73] Chelsea Finn, Pieter Abbeel, and Sergey Levine. Model-agnostic meta-learning for fast adaptation of deep networks. In *International conference on machine learning*, pages 1126–1135. PMLR, 2017.

[74] Aravind Rajeswaran, Chelsea Finn, Sham M Kakade, and Sergey Levine. Meta-learning with implicit gradients. *Advances in neural information processing systems*, 32, 2019.

[75] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. GPT3.int8(): 8-bit matrix multiplication for transformers at scale. *Advances in neural information processing systems*, 35:30318–30332, 2022.

[76] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized llms. *Advances in neural information processing systems*, 36:10088–10115, 2023.

[77] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. OPTQ: Accurate quantization for generative pre-trained transformers. In *The Eleventh International Conference on Learning Representations*, 2023. URL `https://openreview.net/forum?id=tcbBPnfwxS`.

[78] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. Awq: Activation-aware weight quantization for on-device llm compression and acceleration. *Proceedings of Machine Learning and Systems*, 6:87–100, 2024.

[79] Mingjie Sun, Zhuang Liu, Anna Bair, and J Zico Kolter. A simple and effective pruning approach for large language models. In *The Twelfth International Conference on Learning Representations*, 2024. URL `https://openreview.net/forum?id=PxoFut3dWW`.

[80] Elias Frantar and Dan Alistarh. Sparsegpt: Massive language models can be accurately pruned in one-shot. In *International Conference on Machine Learning*, pages 10323–10337. PMLR, 2023.

[81] Rocktim Jyoti Das, Mukul Ranjan, Mingjie Sun, Liqun Ma, and Zhiqiang Shen. Beyond size: How gradients shape pruning decisions in large language models. *arXiv preprint arXiv:2311.04902*, 2023.

[82] Mengzhou Xia, Tianyu Gao, Zhiyuan Zeng, and Danqi Chen. Sheared LLaMA: Accelerating language model pre-training via structured pruning. In *The Twelfth International Conference on Learning Representations*, 2024. URL `https://openreview.net/forum?id=09iOdaeOzp`.

[83] Xinyin Ma, Gongfan Fang, and Xinchao Wang. LLM-Pruner: On the structural pruning of large language models. *Advances in neural information processing systems*, 36:21702–21720, 2023.

[84] Enneng Yang, Li Shen, Guibing Guo, Xingwei Wang, Xiaochun Cao, Jie Zhang, and Dacheng Tao. Model merging in llms, mllms, and beyond: Methods, theories, applications and opportunities. *arXiv preprint arXiv:2408.07666*, 2024.

[85] Charles Goddard, Shamane Siriwardhana, Malikeh Ehghaghi, Luke Meyers, Vladimir Karpukhin, Brian Benedict, Mark McQuade, and Jacob Solawetz. Arcee's mergekit: A toolkit for merging large language models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pages 477–485, 2024.

[86] Shengyu Zhang, Linfeng Dong, Xiaoya Li, Sen Zhang, Xiaofei Sun, Shuhe Wang, Jiwei Li, Runyi Hu, Tianwei Zhang, Guoyin Wang, and Fei Wu. Instruction tuning for large language models: A survey. *ACM Comput. Surv.*, 58(7), 2026. ISSN 0360-0300. doi: 10.1145/3777411. URL `https://doi.org/10.1145/3777411`.

[87] Biao Zhang, Zhongtao Liu, Colin Cherry, and Orhan Firat. When scaling meets LLM finetuning: The effect of data, model and finetuning method. In *The Twelfth International Conference on Learning Representations*, 2024. URL `https://openreview.net/forum?id=5HCnKDeTws`.

[88] Sachin J Chanchani and Ruihong Huang. Composition-contrastive learning for sentence embeddings. *arXiv preprint arXiv:2307.07380*, 2023.

[89] Hicham Badri and Appu Shaji. Half-quadratic quantization of large machine learning models, November 2023. URL `https://mobiusml.github.io/hqq_blog/`.

[90] Shubham Toshniwal, Ivan Moshkov, Sean Narenthiran, Daria Gitman, Fei Jia, and Igor Gitman. Openmathinstruct-1: A 1.8 million math instruction tuning dataset. *Advances in Neural Information Processing Systems*, 37:34737–34774, 2024.

[91] Wasi Uddin Ahmad, Aleksander Ficek, Mehrzad Samadi, Jocelyn Huang, Vahid Noroozi, Somshubra Majumdar, and Boris Ginsburg. Opencodeinstruct: A large-scale instruction tuning dataset for code llms. *arXiv preprint arXiv:2504.04030*, 2025.

[92] Kunyang Li, Jeffrey A Chan Santiago, Sarinda Dhanesh Samarasinghe, Gaowen Liu, and Mubarak Shah. Gvd: Guiding video diffusion model for scalable video distillation. *arXiv preprint arXiv:2507.22360*, 2025.

Appendix Contents

A. Theoretical Analysis

A.1 Convergence Analysis of CAWP 16
A.2 Formal Detectability Advantage of CAWP over Fixed Mappings 18
A.3 Local Proxy-Loss Robustness of FPL 19
A.4 Computational Overhead of FPL 21

B. Method Background

B.1 Configurable Watermark Token Ratio γ 22
B.2 Closed-source Watermark Schemes (KGW & KTH) 22
B.3 Model Modification Techniques 23

C. Experimental Configuration

C.1 Baseline Setting 25
C.2 Training Configurations 25
C.3 Model Modification Settings 26

D. Additional Empirical Results

D.1 Computational Efficiency Analysis 26
D.2 Cross-Distribution and Cross-Architecture Generalization 26
D.3 Ablation on FPL Hyperparameters 26
D.4 Visualization of ROC Curves 27
D.5 Detection Performance under Different FPRs 28
D.6 Overall Detection AUC 28
D.7 Robustness against Paraphrasing Attack 28
D.8 Robustness across Downstream Task Modifications 29
D.9 Comparison with Classifier-based LLM Text Detectors 29
D.10 Cross-Model Attribution across **OPENMARK**-Watermarked Models 30
D.11 Different Embedding and Mapping Models 30
D.12 Distillation Data Scaling Study 30
D.13 Null Calibration and Text Length 30
D.14 Statistical Uncertainty and Independent Training Runs 31
D.15 Decoding Hyperparameter Sweep 32
D.16 Learned Policy Analysis and Fixed Neural Baseline 32
D.17 FPL Alignment and Detector-Refitting Diagnostics 33

E. Limitations 33

F. Impact Statement 34

A Theoretical Analysis

A.1 Convergence Analysis of CAWP

We anchor our proof in established convergence guarantees for distillation-based LLM training, where a student LLM π_θ converges to a fixed teacher π_o under KL divergence minimization. The core innovation of CAWP is introducing a trainable watermark policy M_ϕ that perturbs the teacher distribution. We prove convergence by analyzing this perturbation.

A.1.1 Baseline Distillation Convergence (Fixed Policy M)

For a fixed watermark policy M_{fix}, the loss reduces to standard distillation:

$$\mathcal{L}_{\text{fix}}(\theta) = \text{KL}\left(\pi_o + \delta M_{\text{fix}} \parallel \pi_\theta\right). \tag{7}$$

Under Lipschitz continuity and bounded gradients, gradient descent on θ converges:

$$\lim_{t\to\infty} \|\nabla_\theta \mathcal{L}_{\text{fix}}\| = 0. \tag{8}$$

This serves as the baseline we extend.

A.1.2 CAWP's Joint Optimization

CAWP introduces a trainable policy M_ϕ, leading to the joint loss:

$$\mathcal{L}_{\text{sim}}(\theta, \phi) = \underbrace{\text{KL}\left(\pi_o + \delta M_\phi \parallel \pi_\theta\right)}_{\text{Distillation loss}} + \lambda_2 \underbrace{\mathcal{L}_{\text{norm}}(\phi)}_{\text{Regularizer}}. \tag{9}$$

We analyze the effect of M_ϕ on convergence. Specifically, $\mathcal{L}_{\text{sim}}(\theta, \phi)$ satisfies the following assumptions:

- **Smoothness:** $\nabla\text{KL}(\cdot \parallel \pi_\theta)$ is L_θ-Lipschitz in θ, and ∇M_ϕ is L_ϕ-Lipschitz in ϕ. LLMs and MLPs with smooth activations (e.g., GELU, Tanh) satisfy this.
- **Boundedness:** $\|M_\phi\| \le B_M$ and $\|\nabla_\phi M_\phi\| \le G_M$, since outputs are bounded by the last Tanh layer and gradients are bounded via clipping.
- **Convexity:** $\mathcal{L}_{\text{norm}}$ is convex in ϕ, as ℓ_1 penalties in Eq. (4) enforce convexity.

Given these assumptions, alternating gradient descent on θ and ϕ converges to a stationary point, where M_ϕ learns watermark mappings that perturb π_o without degrading the LLM's performance.

A.1.3 Alternating Gradient Descent Dynamics

In CAWP, optimization alternates between updating θ (distilling to the perturbed teacher) and ϕ (adapting the watermark policy):

$$\theta^{t+1} = \theta^t - \eta_\theta \nabla_\theta \mathcal{L}_{\text{sim}}(\theta^t, \phi^t), \tag{10}$$

$$\phi^{t+1} = \phi^t - \eta_\phi \nabla_\phi \mathcal{L}_{\text{sim}}(\theta^{t+1}, \phi^t). \tag{11}$$

The distillation term encourages π_θ to match the perturbed distribution, while $\mathcal{L}_{\text{norm}}(\phi)$ (e.g., ℓ_1 on policy outputs) prevents M_ϕ from over-perturbing, ensuring watermark detectability without degrading text quality.

Under the smoothness assumption, the loss is L-smooth overall ($L = L_\theta + \delta L_\phi B_M$). Hence,

$$\begin{aligned}\mathcal{L}_{\text{sim}}(\theta^{t+1}, \phi^{t+1}) \le \mathcal{L}_{\text{sim}}(\theta^t, \phi^t) - \left(\frac{\eta_\theta}{2}\|\nabla_\theta \mathcal{L}_{\text{sim}}\|^2 + \frac{\eta_\phi}{2}\|\nabla_\phi \mathcal{L}_{\text{sim}}\|^2\right) \\ + \frac{L\eta_\theta^2}{2}\|\nabla_\theta \mathcal{L}_{\text{sim}}\|^2 + \frac{L\eta_\phi^2}{2}\|\nabla_\phi \mathcal{L}_{\text{sim}}\|^2.\end{aligned} \tag{12}$$

Choosing $\eta_\theta, \eta_\phi \le 1/L$ ensures monotonic decrease:

$$\mathcal{L}_{\text{sim}}(\theta^{t+1}, \phi^{t+1}) \le \mathcal{L}_{\text{sim}}(\theta^t, \phi^t) - c\Big(\|\nabla_\theta \mathcal{L}_{\text{sim}}\|^2 + \|\nabla_\phi \mathcal{L}_{\text{sim}}\|^2\Big), \quad c > 0. \tag{13}$$

A.1.4 Convergence to Stationary Point

Summing the descent inequality over T iterations gives:

$$\sum_{t=0}^{T-1} \left(\|\nabla_\theta \mathcal{L}_{\text{sim}}(\theta^t, \phi^t)\|^2 + \|\nabla_\phi \mathcal{L}_{\text{sim}}(\theta^t, \phi^t)\|^2\right) \le \frac{\mathcal{L}_{\text{sim}}(\theta^0, \phi^0) - \mathcal{L}_{\text{sim}}^*}{cT}, \tag{14}$$

where $\mathcal{L}_{\text{sim}}^*$ is the infimum of the loss. As $T \to \infty$, the gradients vanish:

$$\lim_{t\to\infty} \|\nabla_\theta \mathcal{L}_{\text{sim}}\| = 0, \quad \lim_{t\to\infty} \|\nabla_\phi \mathcal{L}_{\text{sim}}\| = 0. \tag{15}$$

The boundedness of M_ϕ and its gradients ensures that the perturbation δM_ϕ remains controlled, preventing divergence. Moreover, the μ-strong convexity of $\mathcal{L}_{\text{norm}}$ implies that, for fixed θ, the subproblem in ϕ is strongly convex, guaranteeing convergence to a unique minimizer $\phi^*(\theta)$ that balances watermark strength and regularization.

A.2 Formal Detectability Advantage of CAWP over Fixed Watermark Mappings

In this section, we formally show that CAWP achieves a strictly smaller generation-detection inconsistency compared to the baseline (fixed watermark mappings). Our analysis leverages (i) the irreducible non-approximability of hash-like fixed mappings and (ii) standard generalization bounds for the joint hypothesis class of student LLMs and adaptive watermark policies.

Notation. Let $\pi_\theta(y \mid x)$ denote the student LLM, and let $q_{\text{fix}}(y \mid x)$ denote the teacher distribution generated by a fixed watermark mapping M_{fix} (such as KGW/KTH). For CAWP, let $q_\phi(y \mid x)$ denote the teacher distribution generated by the trainable watermark policy M_ϕ.

Define the population KL risks:

$$\mathcal{R}_{\text{fix}}(\theta) = \mathbb{E}_x \,\mathrm{KL}\big(q_{\text{fix}}(\cdot \mid x) \,\|\, \pi_\theta(\cdot \mid x)\big), \tag{16}$$

$$\mathcal{R}_{\text{CAWP}}(\theta, \phi) = \mathbb{E}_x \,\mathrm{KL}\big(q_\phi(\cdot \mid x) \,\|\, \pi_\theta(\cdot \mid x)\big). \tag{17}$$

The optimal risks for the two settings are:

$$\gamma := \inf_\theta \mathcal{R}_{\text{fix}}(\theta), \qquad \varepsilon := \inf_{\theta,\phi} \mathcal{R}_{\text{CAWP}}(\theta, \phi). \tag{18}$$

Assumption 1 (Irreducible error of baseline fixed mappings). The fixed watermark mapping M_{fix} is hash-like, discontinuous, and not aligned with the transformer parameterization. Thus, there exists a constant $\gamma_0 > 0$ such that

$$\forall\, \theta: \quad \mathcal{R}_{\text{fix}}(\theta) \geq \gamma_0, \tag{19}$$

and hence:

$$\gamma = \inf_\theta \mathcal{R}_{\text{fix}}(\theta) \;\geq\; \gamma_0. \tag{20}$$

This corresponds to the empirical mismatch observed in Figure 5 (left), where the student fails to reproduce the fixed green-token distribution.

Assumption 2 (Generalization of CAWP). Let $\widehat{\mathcal{R}}_{\text{CAWP}}(\theta, \phi)$ denote the empirical CAWP loss over N training samples:

$$\widehat{\mathcal{R}}_{\text{CAWP}}(\theta, \phi) = \frac{1}{N} \sum_{i=1}^{N} \mathrm{KL}\big(q_\phi(\cdot \mid x_i) \| \pi_\theta(\cdot \mid x_i)\big). \tag{21}$$

We assume that the joint hypothesis class $\mathcal{H} = \{(\theta, \phi)\}$ has finite capacity (e.g., finite Rademacher complexity), following standard generalization analyses of deep neural networks [70–72]. We do not require the explicit value of $\mathfrak{C}$, only that it is finite, which holds for bounded-parameter networks such as the CAWP policy and the student LLM. Moreover, the KL loss is bounded for all softmax-based language models, as both $q_\phi(\cdot \mid x)$ and $\pi_\theta(\cdot \mid x)$ are strictly positive distributions over a finite vocabulary.

Under these standard conditions, classical uniform-convergence bounds [70] imply that there exists a constant $C > 0$ such that, with probability at least $1 - \delta$,

$$\mathcal{R}_{\text{CAWP}}(\theta, \phi) \leq \widehat{\mathcal{R}}_{\text{CAWP}}(\theta, \phi) + C\sqrt{\frac{\mathfrak{C} + \log(1/\delta)}{N}}. \tag{22}$$

Assumption 3 (Effective optimization of CAWP). Let $(\theta^\star, \phi^\star)$ be an approximate empirical minimizer:

$$\widehat{\mathcal{R}}_{\text{CAWP}}(\theta^\star, \phi^\star) \leq \inf_{\theta,\phi} \widehat{\mathcal{R}}_{\text{CAWP}}(\theta, \phi) + \eta, \tag{23}$$

where η is negligible. Because M_ϕ adapts to the representational capacity of the student LLM, we may assume:

$$\inf_{\theta,\phi} \widehat{\mathcal{R}}_{\text{CAWP}}(\theta, \phi) \approx 0, \qquad \widehat{\mathcal{R}}_{\text{CAWP}}(\theta^\star, \phi^\star) \approx 0. \tag{24}$$

Theorem A.1 (Detectability gain of CAWP)**.** *Under Assumptions 1–3, with probability at least* $1 - \delta$*, the optimal risks satisfy*

$$\gamma \;\geq\; \gamma_0 \qquad \textit{and} \qquad \varepsilon \;\leq\; C\sqrt{\frac{\mathfrak{C} + \log(1/\delta)}{N}}. \tag{25}$$

Consequently, the relative and absolute gaps between CAWP and fixed watermark mappings are bounded by

$$\frac{\varepsilon}{\gamma} \leq \frac{C}{\gamma_0}\sqrt{\frac{\mathfrak{C} + \log(1/\delta)}{N}} \qquad \textit{and} \qquad \gamma - \varepsilon \geq \gamma_0 - C\sqrt{\frac{\mathfrak{C} + \log(1/\delta)}{N}}. \tag{26}$$

The proof is detailed:

Step 1: Lower bound on γ. Immediate from Assumption 1.

Step 2: Upper bound on ε. Applying the generalization bound Equation 22 to $(\theta^\star, \phi^\star)$ gives

$$\mathcal{R}_{\text{CAWP}}(\theta^\star, \phi^\star) \leq \widehat{\mathcal{R}}_{\text{CAWP}}(\theta^\star, \phi^\star) + C\sqrt{\frac{\mathfrak{C} + \log(1/\delta)}{N}}. \tag{27}$$

Using Assumption 3 yields

$$\mathcal{R}_{\text{CAWP}}(\theta^\star, \phi^\star) \leq C\sqrt{\frac{\mathfrak{C} + \log(1/\delta)}{N}}. \tag{28}$$

Since ε is the infimum over all (θ, ϕ),

$$\varepsilon \leq \mathcal{R}_{\text{CAWP}}(\theta^\star, \phi^\star) \leq C\sqrt{\frac{\mathfrak{C} + \log(1/\delta)}{N}}. \tag{29}$$

Step 3: Relative and absolute gaps. Combining $\gamma \geq \gamma_0$ and the above yields

$$\frac{\varepsilon}{\gamma} \leq \frac{C}{\gamma_0}\sqrt{\frac{\mathfrak{C} + \log(1/\delta)}{N}}. \tag{30}$$

Furthermore,

$$\gamma - \varepsilon \geq \gamma_0 - C\sqrt{\frac{\mathfrak{C} + \log(1/\delta)}{N}}. \tag{31}$$

This completes the proof.

Discussion. The bound above shows that the detectability gain of CAWP over fixed watermark mappings is driven by three intuitive factors: (i) a larger irreducible error γ_0 of the fixed mapping (harder-to-learn hash-like patterns), (ii) a larger training sample size N, which shrinks the generalization term, and (iii) a smaller effective capacity $\mathfrak{C}$ of the CAWP model. In other words, CAWP yields the largest improvement precisely in the regime where fixed mappings are intrinsically hard to learn, while CAWP is trained with sufficient data and a well-regularized architecture.

This comparison characterizes the population teacher–student KL mismatch and motivates CAWP's design; detection performance is evaluated empirically in Section 5.

A.3 Local Proxy-Loss Robustness of FPL

The analysis below compares a lower bound on the baseline's worst-case forgetting with an upper bound on FPL's, yielding a local robustness guarantee for the anti-watermark proxy loss within a bounded perturbation radius.

Robustness setting. Let θ denote the parameters of a watermarked student model, and let $\theta' = \theta + \Delta$ be a user-modified model after some fine-tuning, merging, or other downstream adaptation. We

assume the user perturbation is bounded in parameter space, $\|\Delta\| \le R$, where $R > 0$ is a given radius (e.g., induced by model merging or a limited number of fine-tuning steps).

Let $q_{\mathrm{anti}}(\cdot \mid x)$ denote the *anti-watermark teacher* distribution, obtained by subtracting the green-list bias from the original watermark logits (cf. Eq. (6) in the main text). We define the *anti-watermark loss*

$$L_{\mathrm{anti}}(\theta) := \mathbb{E}_{x \sim D_{\mathrm{anti}}}\Big[\mathrm{KL}\big(q_{\mathrm{anti}}(\cdot \mid x) \,\|\, \pi_\theta(\cdot \mid x)\big)\Big], \tag{32}$$

where D_{anti} is a distribution of "anti-watermark" inputs (e.g., adversarial prompts or samples generated using the anti-watermark logits). A smaller value of $L_{\mathrm{anti}}(\theta)$ means that the student model behaves more like the anti-watermark teacher on these hard inputs, i.e., the embedded watermark is weaker. Thus "watermark forgetting" corresponds to a *decrease* in L_{anti}.

For a given radius R, we therefore measure robustness by the worst-case *drop* of L_{anti} under bounded perturbations:

$$\Delta^{\downarrow}_{\max}(\theta; R) := \max_{\|\Delta\| \le R} \Big\{ L_{\mathrm{anti}}(\theta) - L_{\mathrm{anti}}(\theta + \Delta) \Big\}. \tag{33}$$

A model with smaller $\Delta^{\downarrow}_{\max}(\theta; R)$ is more robust, because no admissible user modification can significantly push the model towards the anti-watermark distribution.

Assumption 4 (Smoothness of the anti-watermark loss). $L_{\mathrm{anti}}(\theta)$ is L-smooth, i.e., its gradient is Lipschitz continuous with constant $L > 0$:

$$\big\|\nabla L_{\mathrm{anti}}(\theta_1) - \nabla L_{\mathrm{anti}}(\theta_2)\big\| \le L \,\|\theta_1 - \theta_2\| \quad \forall\, \theta_1, \theta_2. \tag{34}$$

This is a standard assumption in optimization for deep networks.

Assumption 5 (FPL reduces the forgetting gradient). Let θ_{base} be the parameters obtained *without* FPL (e.g., using CAWP or a baseline watermarking scheme), and let θ_{FPL} be the parameters obtained *with* FPL. Define

$$G_{\mathrm{base}} := \big\|\nabla L_{\mathrm{anti}}(\theta_{\mathrm{base}})\big\|, \qquad G_{\mathrm{FPL}} := \big\|\nabla L_{\mathrm{anti}}(\theta_{\mathrm{FPL}})\big\|. \tag{35}$$

Since FPL explicitly penalizes moving the model towards the anti-watermark teacher along the worst forgetting direction (see Eq. (6)), it encourages solutions where the anti-watermark loss is locally flat. We therefore assume that for some $\kappa \in (0, 1)$ we have

$$G_{\mathrm{FPL}} \;\le\; \kappa\, G_{\mathrm{base}}, \tag{36}$$

i.e., FPL reduces the norm of the anti-watermark gradient by a multiplicative factor κ compared to the baseline. This assumption motivates our empirical robustness tests under fine-tuning and merging, where FPL-trained models retain stronger detectability after downstream modifications.

Theorem A.2 (Local anti-watermark proxy robustness)**.** *Under Assumptions 4–5, suppose that the same smoothness constant L applies throughout the radius-R neighborhoods of both checkpoints and that $G_{\mathrm{base}} > 0$. For any checkpoint θ in this comparison, writing $G(\theta) = \|\nabla L_{\mathrm{anti}}(\theta)\|$, we have*

$$RG(\theta) - \frac{L}{2}R^2 \le \Delta^{\downarrow}_{\max}(\theta; R) \le RG(\theta) + \frac{L}{2}R^2. \tag{37}$$

In particular,

$$\Delta^{\downarrow}_{\max}(\theta_{\mathrm{base}}; R) \;\ge\; R\, G_{\mathrm{base}} - \frac{L}{2}R^2, \tag{38}$$

$$\Delta^{\downarrow}_{\max}(\theta_{\mathrm{FPL}}; R) \;\le\; R\, G_{\mathrm{FPL}} + \frac{L}{2}R^2 \;\le\; R\,\kappa G_{\mathrm{base}} + \frac{L}{2}R^2. \tag{39}$$

Subtracting the FPL upper bound from the baseline lower bound yields

$$\Delta^{\downarrow}_{\max}(\theta_{\mathrm{base}}; R) - \Delta^{\downarrow}_{\max}(\theta_{\mathrm{FPL}}; R) \;\ge\; R(1-\kappa)\, G_{\mathrm{base}} - LR^2. \tag{40}$$

Thus FPL strictly reduces this worst-case proxy drop whenever

$$0 < R < \frac{(1-\kappa) G_{\mathrm{base}}}{L}. \tag{41}$$

Here, the gradient-norm reduction κ is given by Assumption 5, which the FPL objective is designed to induce.

Proof. By L-smoothness, throughout the stated neighborhoods,

$$\left|L_{\text{anti}}(\theta+\Delta) - L_{\text{anti}}(\theta) - \nabla L_{\text{anti}}(\theta)^\top \Delta\right| \le \frac{L}{2}\|\Delta\|^2. \tag{42}$$

Consequently, for every $\|\Delta\| \le R$,

$$L_{\text{anti}}(\theta) - L_{\text{anti}}(\theta+\Delta) \le -\nabla L_{\text{anti}}(\theta)^\top \Delta + \frac{L}{2}\|\Delta\|^2. \tag{43}$$

Maximizing over $\|\Delta\| \le R$ and using Cauchy–Schwarz yields

$$\max_{\|\Delta\|\le R} \left\{L_{\text{anti}}(\theta) - L_{\text{anti}}(\theta+\Delta)\right\} \le R\left\|\nabla L_{\text{anti}}(\theta)\right\| + \frac{L}{2}R^2. \tag{44}$$

For $G(\theta) > 0$, evaluate the other side of the smoothness inequality at $\Delta_* = -R\nabla L_{\text{anti}}(\theta)/G(\theta)$:

$$\Delta^{\downarrow}_{\max}(\theta; R) \ge L_{\text{anti}}(\theta) - L_{\text{anti}}(\theta+\Delta_*) \ge RG(\theta) - \frac{L}{2}R^2. \tag{45}$$

If $G(\theta) = 0$, the lower bound also holds since $\Delta = 0$ gives a nonnegative maximum drop. Applying the lower bound to θ_{base} and the upper bound to θ_{FPL}, then using Assumption 5, gives

$$\Delta^{\downarrow}_{\max}(\theta_{\text{base}}; R) - \Delta^{\downarrow}_{\max}(\theta_{\text{FPL}}; R) \ge \left(R\,G_{\text{base}} - \frac{L}{2}R^2\right) - \left(R\,\kappa G_{\text{base}} + \frac{L}{2}R^2\right) \tag{46}$$

$$= R(1-\kappa)\,G_{\text{base}} - LR^2. \tag{47}$$

The right-hand side is positive precisely under the stated local-radius condition. □

Relation to experiments. The guarantee applies where the $O(R)$ gradient benefit dominates the $O(R^2)$ curvature term. Empirically, the controlled FPL ablation keeps unmodified AUC at 0.99 while increasing post-merge AUC from 0.82 to 0.87 with $\beta : 0 \to 5$ (PPL 4.18 to 4.22; Appendix D.3), and the merging and fine-tuning results in Section 5 complement this local analysis on practical modifications.

A.4 Computational Overhead of FPL

In this section, we provide additional details on the computational overhead of the proposed Forgotten Perturbation-aware Learning (FPL). Although the optimization objective in Equation 6 involves second-order derivatives, we show that it can be solved efficiently without computing exact Hessian information.

$$\arg\min_{\pi_\theta} \mathcal{L}_{\text{sim}}(\pi_\theta) + \beta\left(\mathcal{L}_{\text{anti}}(\pi_\theta) - \mathcal{L}_{\text{anti}}\left(\pi_\theta - \alpha\frac{\nabla\mathcal{L}_{\text{anti}}(\pi_\theta)}{\|\nabla\mathcal{L}_{\text{anti}}(\pi_\theta)\|}\right)\right) \tag{48}$$

where $\mathcal{L}_{\text{anti}}(\cdot)$ denotes the forgetting loss on anti-watermarked texts. Intuitively, the second term measures the decrease of $\mathcal{L}_{\text{anti}}$ after one normalized fine-tuning step along the forgetting gradient. By minimizing this gap, the model not only learns a strong watermark but also remains robust to potential forgetting induced by user-side modifications.

To solve this perturbation minimization problem, we consider an iterative gradient method (e.g., SGD). By the chain rule, the update rule is:

$$\pi_\theta^{t+1} = \pi_\theta^t - \eta\left(\nabla\mathcal{L}_{\text{sim}}(\pi_\theta^t) + \beta\left(\nabla\mathcal{L}_{\text{anti}}(\pi_\theta^t) - \nabla\mathcal{L}_{\text{anti}}\left(\pi_\theta^t - \alpha\frac{\nabla\mathcal{L}_{\text{anti}}(\pi_\theta^t)}{\|\nabla\mathcal{L}_{\text{anti}}(\pi_\theta^t)\|}\right)\underbrace{\nabla\left(\pi_\theta^t - \alpha\frac{\nabla\mathcal{L}_{\text{anti}}(\pi_\theta^t)}{\|\nabla\mathcal{L}_{\text{anti}}(\pi_\theta^t)\|}\right)}_{\text{second-order term}}\right)\right) \tag{49}$$

where η is the learning rate. The last factor involves a second-order term (i.e., Hessian information), which is expensive to compute. Following prior work [73, 74], we approximate this second-order term as a constant. The update rule then simplifies to:

$$\pi_\theta^{t+1} = \pi_\theta^t - \eta\left(\nabla\mathcal{L}_{\text{sim}}(\pi_\theta^t) + \beta\left(\nabla\mathcal{L}_{\text{anti}}(\pi_\theta^t) - \nabla\mathcal{L}_{\text{anti}}\left(\pi_\theta^t - \alpha\frac{\nabla\mathcal{L}_{\text{anti}}(\pi_\theta^t)}{\|\nabla\mathcal{L}_{\text{anti}}(\pi_\theta^t)\|}\right)\right)\right) \tag{50}$$

With this approximation, FPL requires only three forward/backward passes per optimization step:

1. A forward pass to compute $\mathcal{L}_{\text{sim}}(\pi_\theta)$ on watermarked text.
2. A forward and backward pass to compute $\nabla\mathcal{L}_{\text{anti}}(\pi_\theta)$ on anti-watermarked text.
3. A forward pass to evaluate $\mathcal{L}_{\text{anti}}(\pi_\theta - \alpha\hat{g})$, where $\hat{g}$ is the normalized forgetting gradient.

Thus, the overhead introduced by FPL is minimal. Moreover, CAWP accelerates convergence during training, making the overall computational cost comparable to baseline methods.

B Method Background

B.1 Configurable Watermark Token Ratio γ

In the main text, the watermark mapping model M is trained to produce logits that are approximately centered around zero, which results in an effective green token ratio close to 0.5. This choice is made for simplicity and does not limit the flexibility of the framework. **OPENMARK** also supports arbitrary watermark token ratios $\gamma \in (0, 1)$.

To target a desired ratio γ, we apply a simple scaling to the raw watermark logits. Let $v_i = M(e_i) \in [-1, 1]^{|\mathcal{V}|}$ denote the watermark logits corresponding to the embedding e_i. We define a transformed logit vector $s_i = [s_{i,j}]_{j=1}^{|\mathcal{V}|}$ as:

$$s_{i,j} = \begin{cases} \frac{1-\gamma}{\gamma}\, v_{i,j}, & v_{i,j} > 0, \\ v_{i,j}, & \text{otherwise.} \end{cases}$$

During training, the normalization objective is applied to s_i in place of $M(e_i)$, encouraging the scaled logits to realize the specified green token proportion γ. The similarity objective $\mathcal{L}_{\text{sim}}$ remains unchanged.

The ratio γ specifies the token partition used during training. At detection time, **OPENMARK** uses the same continuous policy score as in calibration, with the threshold estimated empirically on an independent null set for each configuration (Appendix D.13).

B.2 Closed-source Watermark Schemes (KGW & KTH)

B.2.1 KGW

The KGW watermark introduces a bias into the model's decoding distribution[21]. Given the model output distribution p, the watermarked distribution is defined as

$$f_w^{\text{KGW}} = \text{softmax}(\log p + \delta \cdot h(x_{t-k+1:t}; \xi, \gamma, |\mathcal{V}|)), \tag{51}$$

where $h(\cdot)$ produces a binary vector over the vocabulary that identifies the γ-portion of tokens to which the bias δ is applied. Its output is determined by a secret key ξ and the previous k tokens; when $k > 1$, an additive hash is applied, and when $k = 0$, a fixed green list is used[28].

Detection. Given a sequence x, the detector reconstructs the green list that would have been active at each step and checks whether the produced token belongs to it:

$$S = \sum_{t=k+1}^{\text{len}(x)} h(x_{t-k:t-1})_{x_t}. \tag{52}$$

For normal (unwatermarked) text, the matched tokens should appear only at the rate implied by the green-list proportion γ. If S is noticeably larger than this expected level, it indicates that the generation process systematically preferred green-listed tokens, indicating the presence of the watermark.

B.2.2 KTH

The KTH watermark modifies decoding by selecting the token that best matches a key-dependent score. Given the model distribution p, the watermarked decoder is

$$f_w^{\text{KTH}}(p, x, \xi) = \text{onehot}\left(\arg\max_i \frac{\xi_i^{(\text{len}(x))}}{p_i}, |\mathcal{V}|\right), \tag{53}$$

where the key $\xi = (\xi^{(1)}, \ldots, \xi^{(m)})$ consists of m vectors in $[0,1]^{|\mathcal{V}|}$, each sampled independently. This construction deterministically chooses the token with the largest key-to-probability ratio.

To increase variability across generations from the same prompt, the key is shifted by an offset τ before decoding, producing

$$\xi' = (\xi^{(1+\tau \bmod m)}, \ldots, \xi^{(m+\tau \bmod m)}).$$

A parameter $s \in [1, m]$ specifies the number of allowed shifts, taken as evenly spaced values

$$\tau = \{\, i \cdot \lfloor m/s \rfloor \mid 0 \le i < s \,\}.$$

Larger s broadens the range of possible outputs and increases diversity.

Detection. To evaluate a candidate sequence x, the detector computes the alignment cost

$$d(x, \xi) = \sum_{t=1}^{\mathrm{len}(x)} \log\Big(1 - \xi_{x_t}^{(t)}\Big), \tag{54}$$

where lower values indicate that the text follows the key more closely and is therefore more likely watermarked. The significance of the observed score is assessed using a reference distribution obtained from non-watermarked text, and with a reference set of size T, the resulting p-value is bounded below by $1/(T+1)$.

B.3 Model Modification Techniques

Open-source LLMs are subject to modifications for task adaptation or deployment efficiency. The most prevalent types of such modifications include quantization, pruning, merging, and fine-tuning.

B.3.1 Quantization

Model quantization is an important technique for running large language models on memory-constrained hardware. It reduces the precision of parameters and activations from formats such as FP32 or BF16 to integer representations like INT8 or INT4, helping to lower storage use and speed up inference.

Quantization approaches can be grouped into weight-only quantization and calibration-guided quantization. Weight-only methods apply fixed quantizers directly to model weights without data calibration, such as LLM.int8() [75] and the NF4 format used in QLoRA [76]. Calibration-guided methods optimize quantization parameters using a small calibration set to reduce reconstruction error. HQQ focuses on weight reconstruction, while GPTQ [77] and AWQ [78] further reduce activation-level errors to improve the fidelity of the quantized model.

B.3.2 Pruning

Model pruning serves as a critical technique for optimizing large language models by systematically eliminating unnecessary parameters to achieve computational efficiency and reduced memory footprint. The fundamental principle involves identifying and removing weights that contribute minimally to model performance, thereby creating a more compact representation without substantial accuracy degradation.

Pruning strategies divide into two primary paradigms: unstructured and structured methodologies. In unstructured pruning, individual parameters are evaluated and removed based on their importance, creating irregular sparsity patterns throughout the network. This granular approach enables precise control over sparsity levels but often requires specialized hardware for efficient inference. Conversely, structured pruning eliminates coherent parameter groups—including entire channels, filters, attention mechanisms, or even complete layers—resulting in regular sparsity patterns that align naturally with existing hardware architectures.

The optimization objective in pruning centers on preserving the functional behavior of the original model $f(\cdot; \boldsymbol{\theta})$ after parameter removal. Given a calibration dataset $\mathcal{D}$, the pruning process seeks to identify sparse parameters $\boldsymbol{\theta}'$ that minimize the divergence between the original and compressed models:

$$\min_{\boldsymbol{\theta}'} \sum_{x \in \mathcal{D}} \|f(x; \boldsymbol{\theta}) - f(x; \boldsymbol{\theta}')\|^2, \tag{55}$$

Here, $\boldsymbol{\theta}'$ represents the parameter set after pruning, where numerous weights have been eliminated through zeroing (unstructured) or complete removal (structured).

Contemporary unstructured approaches like WANDA [79], SPARSEGPT [80], and GBLM [81] employ sophisticated scoring mechanisms to assess parameter significance at the individual weight level, enabling high sparsity ratios while maintaining model quality. Meanwhile, structured techniques including SHEARED LLAMA [82] and LLM-PRUNER [83] adopt a holistic approach, jointly optimizing the removal of architectural components to achieve efficient compression while adhering to the reconstruction objective.

B.3.3 Model Merging

Model merging represents an innovative paradigm for synthesizing multiple specialized models into a unified architecture by strategically combining their learned parameters. This approach enables the integration of diverse capabilities acquired through task-specific training, effectively consolidating expertise from different domains into a single deployable model without requiring additional training.

Recent investigations [84, 85] have revealed that parameter-space merging can successfully preserve and even amplify performance across multiple tasks, suggesting that distinct model capabilities can coexist within a shared parameter framework. The theoretical foundation rests on the notion of *task vectors*—the hypothesis that different model competencies manifest as approximately orthogonal directions within the high-dimensional parameter space, enabling their combination through various mathematical operations including weighted averaging and geometric interpolation.

Among the various merging strategies, *Spherical Linear Interpolation* (SLERP) emerges as a mathematically rigorous approach that respects the geometric structure of the parameter space. Unlike naive linear interpolation, SLERP traces the geodesic path between model parameters on the surface of a hypersphere, maintaining constant norm throughout the interpolation process. For two parameter configurations $\boldsymbol{\theta}_1$ and $\boldsymbol{\theta}_2$ separated by angle Ω, the interpolated parameters at position $t \in [0, 1]$ are computed as:

$$\text{SLERP}(\boldsymbol{\theta}_1, \boldsymbol{\theta}_2, t) = \frac{\sin[(1-t)\Omega]}{\sin \Omega}\boldsymbol{\theta}_1 + \frac{\sin[t\Omega]}{\sin \Omega}\boldsymbol{\theta}_2. \tag{56}$$

This spherical interpolation maintains the magnitude relationships inherent in the original models, potentially preserving learned representations more faithfully than alternative methods. When the parameter vectors align perfectly ($\Omega = 0$), the formulation gracefully degenerates to standard linear interpolation, ensuring computational stability across all configurations. The geometric preservation properties of SLERP make it particularly suitable for merging models where maintaining the relative structure of learned features is crucial for performance retention.

B.3.4 Fine-tuning

Model finetuning [86, 87] constitutes a fundamental adaptation methodology that transforms general-purpose language models into specialized systems through targeted training on domain-specific or task-oriented datasets. This technique bridges the gap between broad pretraining objectives and precise downstream requirements, enabling models to acquire specialized knowledge and behaviors absent from their initial training distribution.

The finetuning paradigm encompasses several distinct approaches, notably *Supervised Finetuning (SFT)* and *Instruction Tuning*. SFT directly optimizes model parameters using labeled examples from the target domain, establishing direct mappings between inputs and desired outputs. Instruction tuning extends this framework by training models to interpret and execute natural language directives across heterogeneous tasks, fostering more flexible and generalizable behavior. This instruction-following capability proves essential for developing conversational AI systems that can engage in open-ended dialogue while maintaining alignment with user intentions.

The optimization framework underlying both approaches seeks to minimize prediction error on the adaptation dataset. Starting from pretrained parameters $\boldsymbol{\theta}$, the finetuning process identifies updated parameters $\boldsymbol{\theta}'$ that optimize performance on the target dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$:

$$\min_{\boldsymbol{\theta}'} \frac{1}{N} \sum_{i=1}^{N} \mathcal{L}(f(x_i; \boldsymbol{\theta}'), y_i), \tag{57}$$

Table 5: Watermarking hyperparameter configurations for LLaMA-8B.

Method	Left	Middle	Right
Gu et al. (KGW)	$n = 0, \delta = 1$	$n = 1, \delta = 1$	$n = 1, \delta = 2$
Gu et al. (KTH)	$s = 4$	$s = 2$	$s = 1$
Gloaguen et al. (KGW)	$n = 0, \delta = 1$	$n = 1, \delta = 1$	$n = 1, \delta = 2$
OPENMARK	$\delta = 0.3$	$\delta = 0.5$	$\delta = 1$

Table 6: Watermarking hyperparameter configurations for LLaMA-3.2-3B.

Method	Left	Middle	Right
Gu et al. (KGW)	$n = 0, \delta = 1$	$n = 1, \delta = 1$	$n = 1, \delta = 2$
Gu et al. (KTH)	$s = 4$	$s = 2$	$s = 1$
Gloaguen et al. (KGW)	$n = 0, \delta = 1$	$n = 1, \delta = 1$	$n = 1, \delta = 2$
OPENMARK	$\delta = 0.5$	$\delta = 1$	$\delta = 1.5$

where $\mathcal{L}$ denotes the task-specific loss function measuring the discrepancy between model predictions and ground truth labels.

Contemporary finetuning strategies range from full-model updates to parameter-efficient alternatives. While comprehensive finetuning modifies all model parameters, resource-constrained scenarios benefit from selective adaptation techniques like LoRA (Low-Rank Adaptation) or adapter modules. These methods introduce minimal trainable parameters while freezing the majority of the pretrained model, achieving computational efficiency without compromising adaptation quality. Such parameter-efficient approaches have become increasingly vital as model scales continue to expand, enabling practical deployment of specialized models across diverse computational environments.

C Experimental Configuration

C.1 Baseline Setting

Each table lists the hyperparameter configurations for the four open-source watermarking methods. The columns Left, Middle, and Right indicate the parameter settings used for the leftmost, middle, and rightmost points on each method's curve in the plots. Every **OPENMARK** entry uses $n = 10$ words on all three models; only δ varies across the **OPENMARK** points.

C.2 Training Configurations

In our watermarking pipeline, the semantic embeddings are generated by `compositional-bert-large` [88]. During training, both the student LLM π_θ and the watermark mapping model M are jointly optimized to maximize learnability and detection consistency. We set the regularization weights $\lambda_1 = \lambda_2 = 1$ in the training loss for CAWP. For the FPL component, we use perturbation step size $\alpha = 0.1$ and regularization weight $\beta = 5$. In our ablation study, we further analyze the sensitivity of watermark robustness to variations in these hyperparameters.

All models are fine-tuned on subsets of OpenWebText with different watermark strategies using a batch size of 64 sequences, sequence length of 512 tokens, a maximum learning rate of 1e-5 with cosine decay, and a linear warmup over the first 10% of steps. We use the AdamW optimizer with $(\beta_1, \beta_2) = (0.9, 0.999)$ and no weight decay. For KTH watermark distillation, we follow the same setup except for using a batch size of 128 and a sequence length of 256 tokens to accommodate memory constraints. For the Gloaguen et al. [27]-KGW baseline, we follow their configuration. Starting from a model distilled with KGW, we fine-tune it on OpenWebText for 2,500 steps with a batch size of 64, sequence length of 512 tokens, a learning rate of 1e-5, and the AdamW optimizer. A cosine learning rate schedule is applied with 500 warmup steps. To identify stable parameters, we compute contrastive task vectors based on parameter change before and after fine-tuning, and perform a second-stage distillation restricted to these stable weights. Each training run took approximately 6 hours on 1 NVIDIA B200 192GB GPUs.

Table 7: Watermarking hyperparameter configurations for Phi-2-2.7B.

Method	Left	Middle	Right
Gu et al. (KGW)	$n = 0, \delta = 1$	$n = 1, \delta = 1$	$n = 1, \delta = 2$
Gu et al. (KTH)	$s = 4$	$s = 2$	$s = 1$
Gloaguen et al. (KGW)	$n = 0, \delta = 1$	$n = 1, \delta = 1$	$n = 1, \delta = 2$
OPENMARK	$\delta = 0.5$	$\delta = 1$	$\delta = 1.5$

C.3 Model Modification Settings

To assess robustness, we apply four types of model modifications that simulate real-world user behavior: (1) quantization, including INT8 [75] and GPTQ [77] at 8-bit precision, GPTQ[77] and HQQ [89] at 4-bit; (2) unstructured pruning using WANDA [79] and SparseGPT [80] at 20% and 50% sparsity levels; (3) model merging via SLERP [85], where the watermarked model is interpolated with its non-watermarked base model using mixing ratios from 0.1 to 0.9 following [27]; and (4) full-parameter fine-tuning on the task-specific `OPENMATHINSTRUCT` dataset [90] and `OPENCODEINSTRUCT` dataset [91], reflecting the common use case where LLM users fine-tune open-source models on downstream data to build domain-specific experts. All modifications are implemented following the original settings of their respective methods.

D Additional Empirical Results

D.1 Computational Efficiency Analysis

We provide a detailed comparison of training cost with prior watermarking methods. Our proposed techniques do not incur significant overhead and, in fact, accelerate convergence.

CAWP. The first component, CAWP, introduces only a lightweight trainable MLP as the watermark mapping model M, which contains 1.16M parameters, just 0.0388% of a 3B-parameter LLM. This addition is negligible compared to the base LLM and does not substantially increase computational cost.

FPL. The second component, FPL, involves two additional forward and backward passes per iteration. However, by using a learning-friendly watermark, our method reduces the number of training steps needed to achieve high performance. For example, **OPENMARK** reaches an AUC of 0.997 within 2000 steps, while KGW requires 5000 steps to reach a maximum AUC of 0.991.

Wall-clock Training Time. In practice, the total wall-clock training time remains comparable across methods. For 8B-parameter models trained on $1\times$B200 192GB GPUs, all methods (KGW, KTH, **OPENMARK**) complete training in about 6 hours. Thus, despite the slightly higher per-iteration cost, **OPENMARK** achieves better performance with no additional end-to-end training time.

D.2 Cross-Distribution and Cross-Architecture Generalization

OOD prompts. A practical deployment concern is whether **OPENMARK** remains effective under prompts that differ from the C4 training distribution. We evaluate the C4-trained **OPENMARK** model on Wikipedia prompts using the Figure 6 pipeline, without retraining the student LLM. Figure 7 shows that **OPENMARK** preserves the best detectability-quality trade-off under this distribution shift, suggesting that the learned watermark policy is not tied to C4.

Other architectures. We further evaluate whether **OPENMARK** generalizes beyond LLaMA-family models. Applying our pipeline to GPT-J-6B (a non-LLaMA architecture) using the same settings as Figure 6, **OPENMARK** consistently outperforms baselines: at AUC> 0.99, **OPENMARK** maintains a perplexity of 17.8, while existing methods require at least 21.5.

D.3 Ablation on FPL Hyperparameters

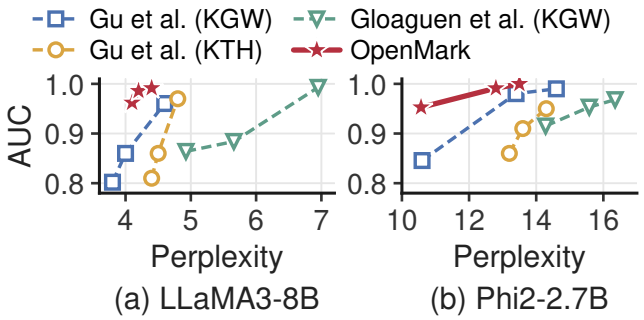


Figure 7: **OpenMark** trained on C4 with LLaMA-3-8B, evaluated on Wikipedia prompts without retraining.

We conduct an ablation study by varying the perturbation step size α (Table 8a) and the regularization weight β (Table 8b) in Equation 6. When both are set to zero, the model reduces to using CAWP alone. In this case, robustness improves slightly over the baseline (Merge AUC $= 0.82$ after merging), indicating CAWP alone can already enhance watermark robustness. Introducing FPL via non-zero β brings modest but consistent gains in robustness (AUC $= 0.87$ at $\alpha = 0.1$, $\beta = 5$), though with a mild increase in perplexity. However, larger values of α or β lead to degraded or unstable results, highlighting the importance of careful hyperparameter tuning to balance robustness and generation quality.

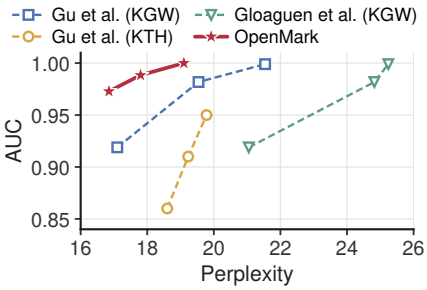


Figure 8: **OpenMark** results on GPT-J-6B, demonstrating generalization beyond LLaMA-family architectures.

Table 8: Ablations on perturbation step size α and regularization weight β. We report AUC and PPL of the unaltered model and AUC after merging ($t = 0.5$) to indicate robustness.

(a) Perturbation step size α.

α	0	**0.1**	1	2	5
AUC	0.99	**0.99**	0.98	0.97	0.95
PPL	4.18	**4.22**	4.30	4.70	5.00
Merge AUC	0.82	**0.87**	0.86	0.83	0.79

(b) Regularization weight β.

β	0	1	**5**	10
AUC	0.99	1.00	**0.99**	0.95
PPL	4.18	4.21	**4.22**	4.52
Merge AUC	0.82	0.85	**0.87**	0.84

D.4 Visualization of ROC Curves

In Figure 9, we visualize the ROC curves for the proposed **OpenMark** and the state-of-the-art method by Gu et al. We select watermarked LLaMA3-8B models with the same level of perplexity (i.e., 4.7). The results indicate that **OpenMark** achieves better watermark detectability while maintaining the same level of generation quality.

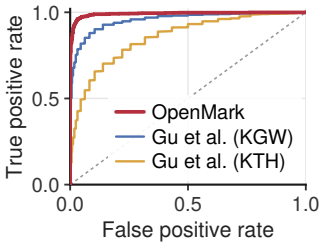


Figure 9: ROC curves for different methods.

Specifically, the ROC curve of **OpenMark** closely follows the top-left corner, indicating a higher true positive rate (TPR) across nearly all false positive rate (FPR) thresholds. In contrast, the methods by Gu et al. show noticeably lower TPRs, particularly at low FPR regions, which suggests a less reliable distinction between watermarked and non-watermarked texts under tight detection constraints. This improvement is especially evident in the early phase of the curve (e.g., FPR < 0.1), where **OpenMark** already achieves near-perfect detection performance, while the baseline methods lag behind. These findings further confirm that **OpenMark** not only embeds robust watermark patterns

but also enables more confident detection, even at low error tolerance levels, making it more suitable for security-critical applications.

D.5 Detection Performance under Different FPRs

We further evaluate detection robustness by measuring the true positive rate (TPR) at different false positive rate (FPR) levels. This complements the AUC metric and highlights performance in stricter detection regimes. As shown in Table 9, **OpenMark** consistently outperforms prior methods, especially under very low FPRs (e.g., 0.1%), which are critical for practical deployment.

Table 9: Comparison of detection performance (TPR at different FPR levels) on LLaMA-3-8B. **OpenMark** demonstrates significantly stronger detection under stringent low-FPR conditions.

Method	TPR@0.1% ↑	TPR@1% ↑	TPR@10% ↑
Gu et al. (KTH)	25.8%	53.6%	84.0%
Gu et al. (KGW)	54.2%	82.7%	97.5%
OpenMark	**78.1%**	**92.3%**	**99.5%**

D.6 Overall Detection AUC

We compared our **OpenMark** and baselines in terms of detection AUC and perplexity in Figure 10, the results show that our method **OpenMark** can achieve the best AUC while maintaining the lowest perplexity.

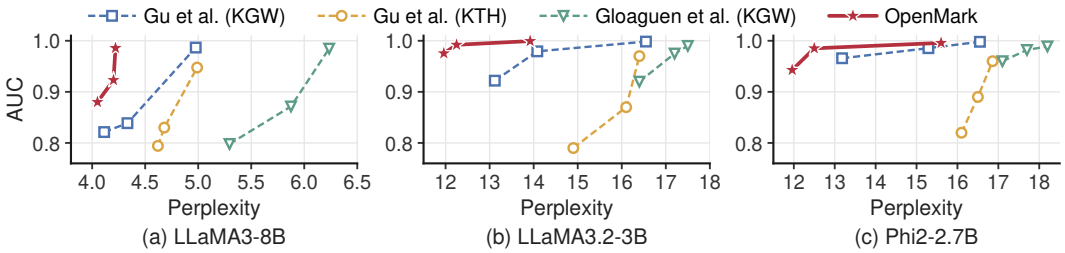


Figure 10: Effectiveness comparison of different open-source LLMs text watermarks, in terms of detection AUC and median PPL on three LLMs. Better watermark detectability and generation quality are indicated by higher AUC and lower PPL, as shown by lines closer to the upper left corner.

D.7 Robustness against Paraphrasing Attack

We evaluate the robustness of our watermarking method under paraphrasing using DIPPER[45], a controllable paraphraser that rewrites text while preserving semantics in Table 10. We use two settings: DIPPER-1, with lexical diversity set to 60 and order diversity 0; and DIPPER-2, with lexical diversity 60 and order diversity 20. **OpenMark** achieves AUC 0.90 and 0.87 under the two settings, versus 0.86 and 0.84 for the strongest baseline, Gu et al.-KGW. We further report TPR at 1% FPR under stronger rewriting attacks below.

Table 10: AUC under DIPPER attacks.

Watermark Method	DIPPER1	DIPPER2
Gu et al. [24]-KTH	0.82	0.79
Gu et al. [24]-KGW	0.86	0.84
Gloaguen et al. [27]-KGW	0.80	0.74
OpenMark (Ours)	**0.90**	**0.87**

Stronger rewriting attacks. We evaluate three attacks on 200-token generations from the released LLaMA3-8B model, using each method's human-C4-calibrated threshold at nominal 1% FPR. For independent paraphrasing, Qwen3.5-9B rewrites each passage once with an instruction to preserve meaning. Repeated paraphrasing applies the same model for three consecutive rounds. For watermark-aware rewriting, the attacker receives each method's private watermark artifact (**OPENMARK**'s detector or KGW's hash key and tokenizer), generates eight diverse candidates, and selects the lowest-scoring one. This oracle attack grants stronger access than the deployment threat model.

Table 11: TPR at nominal 1% FPR under stronger text rewriting. The final row is a detector-oracle stress test.

Attack	Gu et al.-KGW	**OPENMARK**
Independent paraphrasing	0.25	0.32
Repeated paraphrasing, three rounds	0.12	0.17
Watermark-aware rewriting, eight candidates	0.00	0.03

OPENMARK retains higher TPR than Gu et al.-KGW under all three attacks, including the detector-oracle setting that exceeds our threat model.

D.8 Robustness against Model Modifications across Downstream Tasks

We conducted experiments to evaluate FPL's effectiveness on more diverse downstream tasks. Specifically, we fine-tuned on the Alpaca dataset to further examine watermark robustness beyond code and math domains. As shown in Table 12, our method consistently outperforms the best baseline (Gloaguen et al. with KGW), with **OPENMARK**'s relative improvements shown in parentheses. At 1% FPR, **OPENMARK**'s advantage over Gloaguen et al.-KGW grows from +2.8% after 300 steps to +16.5% after 1,500 steps (TPR 54.3% and 16.9%), indicating slower decay under longer fine-tuning.

Table 12: Performance of watermarked LLaMA-3-8B under different fine-tuning steps s. The values in parentheses (prefixed with $+$) indicate the relative improvement compared to the baseline.

Step s	AUC	TPR@1%	TPR@10%
300	0.91 (+0.02)	54.3% (+2.8%)	72.5% (+6.0%)
600	0.87 (+0.04)	49.4% (+4.0%)	69.9% (+2.9%)
900	0.85 (+0.02)	40.5% (+9.1%)	60.2% (+6.9%)
1200	0.81 (+0.04)	26.8% (+13.0%)	44.7% (+13.6%)
1500	0.79 (+0.05)	16.9% (+16.5%)	33.3% (+21.0%)

D.9 Compared with Classifier-based LLM text Detectors

One distinct advantage of watermarking over classifier-based detectors is its ability to attribute text to a specific model, rather than just distinguishing LLM and human text. We evaluated **OPENMARK** using the open-source classifier SuperAnnotate/ai-detector, which flagged 90.6% of **OPENMARK**-generated texts as AI-generated. It also flagged 90% of outputs from non-watermarked and differently watermarked LLMs (Qwen-4B/8B, GPT-6B, LLaMA3-3B) as LLM-generated. To further analyze this, we fine-tuned a binary RoBERTa classifier on texts from LLaMA3-8B and human texts. It detected 60–80% of outputs from other LLMs (Qwen-4B/8B, GPT-6B, LLaMA3-3B) as LLaMA3-8B-generated. This shows that classifier-based methods primarily learn to distinguish LLM vs. human text, not between different LLMs. In contrast, our **OPENMARK** watermark enables model-level attribution. Specifically, we used the watermark policy model trained with LLaMA3-8B (student LLM) to detect outputs from other LLMs. The false positive rates remained low: 0.8% (Qwen-4B), 1.0% (Qwen-8B), 1.1% (GPT-6B), and 0.6% (LLaMA3-3B). This highlights **OPENMARK**'s precision in determining whether a text was generated by a specific model.

D.10 Cross-Model Attribution across OPENMARK-Watermarked Models

We evaluate each detector M_i on outputs of all three independently watermarked generators G_j. A matched pair ($i = j$) measures detection of the model's own outputs. A mismatched pair ($i \neq j$) measures how often the detector flags another watermarked model's outputs. Section 5.2, Table 3, reports all nine rates.

D.11 Different Embedding Model and Watermark Mapping Model

To further validate the generality of our method, we supplemented the experiments with two additional embedding models: `thenlper/gte-large` and `intfloat/e5-large-v2`. Results in Table 13 show that our method maintains strong detection performance across different embeddings, consistently achieving near-perfect AUC and high TPR scores.

Table 13: Performance of **OPENMARK** watermarking with LLaMA-3.2-3B under different embedding models.

Embedding Model	AUC	TPR@1%	TPR@10%
`thenlper/gte-large`	0.997	95.2%	99.2%
`intfloat/e5-large-v2`	0.994	94.7%	99.0%

We also compared different architectures for the watermark mapping model M. An MLP yields the best performance, as it can effectively exploit the full semantic embedding. By contrast, convolutional neural networks (CNNs) perform significantly worse, since semantic embeddings are single dense vectors without spatial or sequential structure for convolution to leverage. Thus, we adopt the MLP design in our framework.

D.12 Distillation Data Scaling Study

Recent work by [27] shows that watermark robustness can be improved by substantially enlarging the distillation dataset [92]. In Table 2, **OPENMARK** uses a 64 million tokens distillation corpus to maintain the trade-off between watermark robustness and model fidelity. To examine how **OPENMARK** benefits from data scaling, we conduct additional experiments where the distillation corpus is expanded from 32M to 160M tokens. Table 14 summarizes the results under two robustness settings: (1) model merging (ratio=0.5) and (2) 1500-step fine-tuning on OpenMathInstruct [90]. We observe consistent improvements in TPR@5% as the distillation data increases. For example, TPR@5% FPR improves from 0.49 to 0.69 under merge-0.5, and from 0.29 to 0.72 under fine-tuning. The perplexity increases only moderately (4.0 to 5.3), indicating that data scaling does not substantially harm model quality.

Table 14: Effect of distillation data scaling on watermark robustness. Larger distillation datasets improve TPR@5%FPR under both merge-0.5 and FT-1500 settings, with only moderate PPL increase.

Modification	**32M**	**64M**	**96M**	**128M**	**160M**
Merge-0.5	0.49	0.57	0.63	0.67	0.69
FT-1500	0.29	0.37	0.50	0.63	0.72
PPL	4.0	4.2	4.7	5.1	5.3

D.13 Null Calibration and Text Length

The main null consists of 5,000 held-out human-C4 continuations. Calibration uses a separate, non-overlapping set of 200-token human-C4 texts. For each **OPENMARK** detector, we set τ_{C4} to the empirical 99th-percentile calibration score and freeze it before evaluation. The same threshold is applied to all null sources below without recalibration.

The realized FPRs stay close to the nominal 1% across all three null sources, so the human-C4 threshold transfers without source-specific recalibration. With the 200-token threshold frozen, longer

Table 15: Realized FPR on disjoint 200-token null sources at the same frozen human-C4 threshold. **OPENMARK**'s watermarked outputs have TPR 91.8% at this threshold.

Evaluation null	Realized FPR
Held-out human C4	1.06%
Unwatermarked same-base generations	1.28%
Qwen3-4B generations	0.82%

texts yield lower FPR and higher TPR (Table 16), since they provide more watermark evidence and lower null-score variance; length-specific calibration can target the nominal FPR at each length.

Table 16: Length transfer of a single detector and threshold calibrated on 200-token human-C4 text.

Text length (tokens)	100	200	300	500
Realized FPR	1.42%	1.06%	0.72%	0.38%
TPR	86.3%	91.8%	94.1%	96.6%

For shorter texts, we rescore 200 watermarked outputs of **OPENMARK** trained on Granite-4.2-3B, together with 500 calibration and 500 test human-C4 texts, each truncated to its first L tokens and scored without the prompt. The threshold is calibrated at each length for 1% FPR (Table 17). Because the first ten words serve as context, a text of L tokens has about $L - 12$ scored tokens. AUC is 0.977 at 50 tokens and 0.997 at 100 tokens. At the per-length threshold, TPR reaches 93.5% at 100 tokens and 71.5% at 50 tokens, while the realized FPR stays at or below 1.2%; the 200-token threshold, in contrast, admits 8.2% and 22.4% FPR at 50 and 25 tokens. A 10-token text has fewer than ten words and thus no scored token; scoring with a 3-word context instead gives AUC 0.739 and TPR 9.5% at 10 tokens, and AUC 0.877 and TPR 28.5% at 20 tokens.

Table 17: Short-text detection with **OPENMARK** on Granite-4.2-3B, scored on the text alone with a threshold calibrated at each length.

Text length (tokens)	25	50	100	200
Scored tokens (mean)	12.7	37.7	87.7	187.7
AUC	0.888	0.977	0.997	1.000
TPR	30.0%	71.5%	93.5%	100%
Realized FPR	1.2%	0.6%	0.2%	0.2%
FPR at the 200-token threshold	22.4%	8.2%	1.4%	0.2%

Table 18 compares the same length-specific detection task with Gu et al.-KGW. The 10-token row uses a 3-word context.

D.14 Statistical Uncertainty and Independent Training Runs

For the LLaMA3-8B results in Table 19, we report 95% prompt-bootstrap confidence intervals computed with 1,000 resamples while holding the independently calibrated detection threshold fixed. These intervals capture evaluation-sample uncertainty for a trained checkpoint; training randomness is evaluated separately below.

To assess training randomness, we train three independently watermarked LLaMA3-8B models. Their unmodified TPR@1% FPR is 0.907 ± 0.015 and PPL is 4.37 ± 0.15 (mean $\pm$ standard deviation), consistent with the main-table checkpoint $(0.920, 4.25)$. Across the remaining main-table cells, bootstrap CI half-widths are at most 0.015 for TPR and 0.24 for median PPL.

Per-model PPL comparison. Table 20 gives the reported **OPENMARK** and Gu et al.-KGW points used for the 20.5% average PPL reduction. For each model, the baseline is the Gu et al.-KGW operating point closest in TPR@1% FPR, at which **OPENMARK** achieves higher TPR. Each

Table 18: Detection on short texts from Granite-4.2-3B. TPR is measured at 1% FPR with a threshold calibrated for each length.

Tokens	Gu et al.-KGW		**OPENMARK**	
	AUC	TPR@1%	AUC	TPR@1%
10	0.667	0.015	0.739	0.095
25	0.758	0.205	0.888	0.300
50	0.936	0.465	0.977	0.715
100	0.971	0.775	0.997	0.935
200	0.983	0.870	1.000	1.000

Table 19: LLaMA3-8B point estimates and 95% prompt-bootstrap intervals reported for the main evaluation. TPR uses the nominal 1% FPR threshold.

Result	Estimate	95% CI
Unmodified TPR	0.920	[0.911, 0.928]
Unmodified median PPL	4.25	[4.17, 4.34]
Merge $t = 0.5$, TPR	0.280	[0.268, 0.293]
OpenMath, 300 steps, TPR	0.430	[0.416, 0.444]
OpenMath, 1,500 steps, TPR	0.170	[0.160, 0.181]

reduction is $(\mathrm{PPL}_{\mathrm{KGW}} - \mathrm{PPL}_{\textbf{OPENMARK}})/\mathrm{PPL}_{\mathrm{KGW}}$, and 20.5% is the arithmetic mean of the three per-model reductions.

Table 20: Reported comparison points underlying the average PPL reduction. The baseline is Gu et al.-KGW for all three models; TPR is at 1% FPR.

	OPENMARK		Gu et al.-KGW		
Model	TPR	PPL	TPR	PPL	Reduction
LLaMA3-8B	0.92	4.25	0.82	5.00	15.0%
LLaMA3.2-3B	0.91	12.40	0.90	17.10	27.5%
Phi2-2.7B	0.80	12.35	0.75	15.25	19.0%
Mean	–	–	–	–	20.5%

D.15 Decoding Hyperparameter Sweep

We evaluate the same **OPENMARK** checkpoint and detector under greedy decoding and sampling temperatures 0.2, 0.5, 1.0, and 1.5, using the main evaluation protocol. The threshold is calibrated once on human C4 and frozen across all settings. TPR ranges from 0.85 to 0.92 (Table 21), indicating stable detectability across decoding settings.

D.16 Learned Policy Analysis and Fixed Neural Baseline

We analyze watermark logits over 14,000 evaluation prefixes. Across corpus frequency, token length, word-initial status, and token type, aggregate differences between promoted and suppressed tokens are on the order of 10^{-3}. The correlation of a token's average policy value with its corpus frequency is only -0.058. These findings support a context-dependent policy rather than a fixed lexical preference.

Table 22 compares part-of-speech distributions, WordNet supersense distributions, and centroids in the LLM input-embedding space with random token splits over 200 draws. The policy split is comparable to random splits on all three measures, indicating no systematic semantic or syntactic bias in the tokens the policy promotes.

To distinguish co-adaptation from neural parameterization, we compare against SIR [56] under the same open-source distillation setting. The fixed neural policy gives TPR@1% FPR 0.86 with PPL

Table 21: Detection under different decoders with the same private detector and frozen threshold.

Decoding	Greedy	$T = 0.2$	$T = 0.5$	$T = 1.0$	$T = 1.5$
TPR at nominal 1% FPR	0.89	0.85	0.89	0.92	0.90

Table 22: Aggregate distances between promoted and suppressed tokens. Random-split values are mean $\pm$ standard deviation across 200 draws.

Comparison	Policy split	Random split
Part-of-speech distributions	0.010	0.012 ± 0.003
WordNet supersense distributions	0.029	0.025 ± 0.003
Embedding-space centroids	0.001	0.0014 ± 0.0000

4.6, compared with CAWP's 0.92 with PPL 4.2. At 0.1% FPR, the respective TPRs are 0.64 and 0.78. This comparison supports the benefit of learning the policy jointly with the student rather than merely replacing a hash with a neural mapping.

D.17 FPL Alignment and Detector-Refitting Diagnostics

Alignment with ordinary fine-tuning. Let $\Delta\theta_s = \theta_s - \theta_{\textsc{OpenMark}}$ be the cumulative fine-tuning update, and define the erasing direction at the released checkpoint as

$$\hat{d} = -\frac{\nabla L_{\text{anti}}(\theta_{\textsc{OpenMark}})}{\|\nabla L_{\text{anti}}(\theta_{\textsc{OpenMark}})\|_2}. \tag{58}$$

Table 23 compares their cosine with the RMS cosine magnitude expected for an independent Gaussian direction of the same dimensionality. Although most of the update serves task adaptation, its alignment with the erasing direction exceeds the random reference by approximately 160–440 times, indicating that ordinary fine-tuning contains a systematic component along the direction FPL defends.

Table 23: Alignment of cumulative fine-tuning updates with the anti-watermark erasing direction.

Step s	300	900	1,500
Cosine with $\hat{d}$	0.0048	0.0021	0.0018
Random-direction RMS	0.000011	0.000011	0.000011
Ratio (approximate)	440×	190×	160×

Erasure versus detector mismatch. User modification can reintroduce mismatch with the frozen detector. To diagnose its contribution, we freeze the modified student, original teacher, and encoder, and refit only M, initialized from the original detector, on a separate OpenWebText split using the CAWP policy objective. We independently recalibrate the refitted detector on human C4 before testing on held-out generations. This owner-side diagnostic is separate from the deployed protocol, which keeps the detector frozen.

The recovered fractions of the original TPR drop are $(0.34 - 0.28)/(0.92 - 0.28) \approx 9.4\%$ and $(0.29 - 0.17)/(0.92 - 0.17) = 16\%$. Since refitting recovers only a small fraction of the drop, detection loss mainly reflects erasure of the watermark signal rather than detector mismatch, which is the failure mode FPL directly targets.

E Limitations

Platform and agent scope. **OpenMarkX** provides built-in model workflows and extension interfaces, but support depends on the chosen adapter, watermark method, and evaluation stage. Additional integration is required for arbitrary new adapters. The current agent workflow detects generated final answers, not complete action trajectories; the multimodal demonstrations use compatible methods

Table 24: TPR at nominal 1% FPR before and after detector refitting. The unmodified reference TPR is 0.92.

Modification	Before refitting	After refitting
Merge $t = 0.5$	0.28	0.34
OpenMath, 1,500 steps	0.17	0.29

rather than the full **OPENMARK** procedure in every modality. These demonstrations do not establish robustness for every method–model–modification combination.

Provenance and forgery. **OPENMARK** targets model-level provenance; user-level attribution and cryptographically unforgeable watermarking [58] are complementary directions.

Private artifacts and extraction. Our deployment assumes the teacher π_o and the policy M remain private; formal guarantees against policy extraction are left to future work.

Bounded robustness. Since a user can arbitrarily overwrite the released weights, no weight-based watermark can survive unbounded modification. **OPENMARK** slows degradation under practical modifications, but detection still decreases under heavier merging, longer fine-tuning, and strong rewriting (Section 5, Appendix D.7). Our theoretical robustness result is local and stated for the anti-watermark proxy loss.

F Impact Statement

This work advances generation watermarking through the **OPENMARKX** platform and the **OPENMARK** method. The platform supports configurable evaluation of open model and agent outputs, while the method improves text-watermark detectability and robustness under the reported protocols. The modality-specific workflows broaden the kinds of generated outputs that can be studied; detection remains statistical evidence under a specified key and protocol. We do not identify any direct or immediate ethical concerns specific to this work.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and introduction's claims about improved learnability (CAWP) and robustness (FPL) match the theoretical analysis in Appendix A and empirical results in Section 5.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: Limitations are discussed in Appendix E.

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
   - While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [Yes]

Justification: All assumptions and proofs (CAWP convergence, detectability advantage, FPL robustness, FPL overhead) are given in Appendix A.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: [Yes]

   Justification: The method, datasets, models, baselines, and hyperparameters are described in Section 4–5 and Appendix C.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
   - If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
   - Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
   - While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
     (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
     (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
     (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
     (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: Code is released at `https://github.com/loopleap-lab/OpenMarkX`, and all datasets and pretrained models used are publicly available.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: Training and evaluation details are reported in Section 5 and Appendix C.2.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: Appendix D.14 reports 95% intervals from 1,000 prompt-bootstrap resamples with the calibrated threshold fixed, separately from mean and standard deviation over three independent watermark-training seeds.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).

- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [Yes]

   Justification: Compute resources and runtime are reported in Appendix D.1 and Appendix C.2.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

   Justification: The research uses only public datasets and open-source models, and conforms to the NeurIPS Code of Ethics.

   Guidelines:

   - The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
   - The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [Yes]

    Justification: Societal impact is discussed in the Impact Statement (Appendix F).

    Guidelines:

    - The answer [N/A] means that there is no societal impact of the work performed.
    - If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.

- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

    Answer: [N/A]

    Justification: This work proposes a watermarking technique aimed at improving traceability of open-source LLMs and does not release any high-risk dataset or generative model.

    Guidelines:

    - The answer [N/A] means that the paper poses no such risks.
    - Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
    - Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
    - We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: [Yes]

    Justification: All used models (LLaMA3-8B, LLaMA3.2-3B, Phi2-2.7B, Llama-3.1-8B, Granite-4.2-3B, GPT-J-6B, Qwen3-4B, Qwen3.5-9B, MedGemma-1.5-4B, Taming Transformers, GeoPT, gte-large, e5-large-v2), datasets (C4, OpenWebText, OpenMathInstruct, OpenCodeInstruct, Alpaca), and baseline implementations are publicly released by their creators and credited in the paper. They are used under their original licenses, e.g., the Meta Llama Community Licenses, MIT for Phi-2, Apache 2.0 for Granite, GPT-J and Qwen3, and ODC-BY for C4.

    Guidelines:

    - The answer [N/A] means that the paper does not use existing assets.
    - The authors should cite the original paper that produced the code package or dataset.
    - The authors should state which version of the asset is used and, if possible, include a URL.

- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [Yes]

    Justification: The code at `https://github.com/loopleap-lab/OpenMarkX` includes documentation for training and evaluating **OPENMARK**-watermarked models.

    Guidelines:

    - The answer [N/A] means that the paper does not release new assets.
    - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
    - The paper should discuss whether and how consent was obtained from people whose asset is used.
    - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [N/A]

    Justification: The paper does not involve crowdsourcing or human subjects research.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
    - According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [N/A]

    Justification: The paper does not involve human subjects.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.

- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

    Answer: [Yes]

    Justification: Open-source LLMs (LLaMA3-8B, LLaMA3.2-3B, Phi2-2.7B) are the watermarking targets and are described throughout Section 4–5.

    Guidelines:

    - The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
    - Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.